

Designing Extensible Command-Line Systems

A pedagogical study of reusable architecture patterns in Cobra

2026-08-16

Contents

Preface	1
What this book teaches	1
The running example: <code>forge</code>	1
Notation	2
Contents at a glance	2
How to read the book	3
Evidence discipline	3
 Chapter 1 - Build a Semantic Command Model	 4
Learning objectives	4
1.1 Motivation: why a table of handlers stops scaling	4
1.2 The command tree as a semantic model	5
1.3 Projections: one model, several interpreters	7
1.4 Root-owned dispatch and leaf-owned action	9
1.5 Hierarchical scope: persistent flags, local flags, and shadowing	12
1.6 A complete worked design review	13
1.7 Limits of the command-tree pattern	14
1.8 Chapter summary	14
Exercises	14
Cobra source map for Chapter 1	15
 Chapter 2 - Execute with Explicit Lifecycles and Contracts	 16
Learning objectives	16
2.1 Motivation: a handler call is not an execution model	16
2.2 A staged execution pipeline	17
2.3 Hooks as ancestry-aware middleware	18
2.4 Positional validation as first-class functions	20
2.5 Declarative relationships among flags	22
2.6 Full worked trace: <code>forge deploy</code>	24
2.7 Failure modes and design tensions	25
2.8 Chapter summary	26
Exercises	26
Cobra source map for Chapter 2	27
 Chapter 3 - Derive Assistance from the Model	 28
Learning objectives	28
3.1 Motivation: assistance is executable behavior	28
3.2 Late-bound synthetic affordances	29
3.3 Completion as an in-band protocol	30
3.4 The completion API: candidates plus directives	31
3.5 Protocol hygiene	34
3.6 Constraint metadata drives both rejection and guidance	35
3.7 Help and documentation as projections	36
3.8 Design tensions	37
3.9 Chapter summary	37
Exercises	38
Cobra source map for Chapter 3	39

Chapter 4 - Testable Boundaries and Ownership-Safe Evolution	40
Learning objectives	40
4.1 Motivation: process defaults are hidden dependencies	40
4.2 An injectable execution environment	41
4.3 Process defaults versus constructor-enforced dependencies	44
4.4 Ownership: a slice value is not an owned array	45
4.5 Failure-derived lesson: completion mutated <code>os.Args</code>	46
4.6 Mutable graphs, execution snapshots, and compatibility	47
4.7 Contract-oriented testing	48
4.8 Capstone: assembling the forge architecture	50
4.9 Chapter summary	50
Exercises	51
Cobra source map for Chapter 4	52
Appendix A - Glossary	53
Appendix B - Selected Exercise Sketches	57
Chapter 1	57
Chapter 2	57
Chapter 3	58
Chapter 4	58
Appendix C - Cobra Source Map and Evidence Notes	59
Snapshot	59
Primary files	59
Primary tests	60
Failure-derived evidence	60
Interpretation limits	60

Preface

A command-line program can begin as a single main function, a few conditionals, and a call into application code. That design is often adequate until the interface acquires nested commands, inherited options, generated help, shell completion, validation rules, tests, plugins, and compatibility requirements. At that point the command line is no longer a thin wrapper around the application. It is a small language, and the program needs an architecture for representing, interpreting, and evolving that language.

This book studies that architecture through [Cobra](#), a Go library for command-line applications. The goal is not to reproduce Cobra's user guide. The goal is to extract design ideas that can be understood, tested, and reused in other frameworks and languages.

The source study is pinned to Cobra commit [adbc8813901bba65827259daa8e22ff94ec1f30e](#), dated 2026-07-11. The book also uses the failure and repair recorded in commit [746ef07158728502482cea9f880a6f4b21ef29a9](#) to teach ownership of borrowed slices. Source links throughout the text point to the pinned snapshot so that the evidence does not silently move as the repository evolves.

What this book teaches

By the end of the four chapters, you should be able to:

1. model a hierarchical command interface as a semantic tree rather than a collection of unrelated handlers;
2. separate root-owned dispatch from leaf-owned execution;
3. define a deterministic lifecycle with ancestry-aware hooks;
4. express argument and flag constraints as composable, declarative contracts;
5. derive help, completion, and documentation from the same semantic model;
6. design completion as a protocol between the shell and the executable;
7. make process-facing dependencies injectable for tests and embedded hosts;
8. reason about slice ownership, shared backing arrays, and compatibility-sensitive evolution.

Each major concept follows the same sequence: a motivating problem, a precise definition, a concrete worked example, a counterexample or failure mode, and exercises. The examples use Go-like APIs because Cobra is written in Go, but the abstractions are language-independent.

Fundamentals - What is a command-line language? A language is a system of symbols and rules that lets a user express an intention. In a CLI, command names, positional arguments, flags, aliases, and separators form the syntax. Routing, validation, and execution give that syntax meaning. Calling the interface a language is not metaphorical excess; it explains why parsing, semantics, diagnostics, tooling, and compatibility all matter.

The running example: forge

The book uses a fictional deployment tool named `forge`. Its initial interface is:

```
forge
├─ project
│   └─ init [directory]
│       └─ inspect
└─ deploy <environment>
```

```
└─ config
  └─ get <key>
  └─ set <key> <value>
```

The root command has persistent flags that apply to descendants:

```
--config <path>      configuration file
--profile <name>     named credentials/profile
--verbose           verbose diagnostics
```

The deploy command later gains flags such as:

```
--image <reference>  deploy a container image
--artifact <path>    deploy a local artifact
--region <name>      target region
--dry-run           plan without applying
--json             emit JSON
--yaml            emit YAML
```

A representative invocation is:

```
$ forge --profile team-a deploy production \
  --image registry.example/api:v42 \
  --region us-east-1 \
  --json
```

The same example will be routed in Chapter 1, executed and validated in Chapter 2, completed and documented in Chapter 3, and tested through injected process boundaries in Chapter 4. Reusing one interface makes it possible to see how the patterns reinforce one another.

Notation

We will use the following notation when a mathematical statement is clearer than prose:

- $T = (V, E, r)$ is a rooted command tree with nodes V , parent-child edges E , and root r .
- $\text{path}(v)$ is the sequence of nodes from r to node v .
- argv is the sequence of command-line tokens supplied to an execution.
- $L(v)$ is the set of flags declared locally on command v .
- $P(v)$ is the set of flags declared persistent on command v .
- $E(v)$ is the effective flag view at v after inheritance and shadowing.
- S is the set of flags actually selected by the user in one invocation.
- A function signature of the form $f : A \rightarrow B$ means that f maps values from domain A to values in codomain B .

The notation is used to sharpen an invariant, not to turn framework design into abstract algebra for its own sake. Every formula is followed by a concrete example.

Contents at a glance

1. **Build a Semantic Command Model** - command trees, projections, root-owned dispatch, and hierarchical flag scope.
2. **Execute with Explicit Lifecycles and Contracts** - lifecycle phases, hooks, validators, and flag-group mathematics.
3. **Derive Assistance from the Model** - late-bound defaults, shell completion as a protocol, and generated documentation.

4. **Testable Boundaries and Ownership-Safe Evolution** - injected process dependencies, slice ownership, compatibility, and testing.

How to read the book

Chapter 1 builds the semantic model. Chapter 2 turns that model into a predictable execution machine. Chapter 3 treats help, completion, and documentation as derived behavior. Chapter 4 concentrates on boundaries, testing, ownership, and safe evolution. The chapters are cumulative: Chapter 3 assumes the tree and execution vocabulary from Chapters 1 and 2, while Chapter 4 tests the whole system.

Readers who already know Cobra can still benefit from reading in order. Familiar APIs are deliberately renamed in architectural terms - *semantic tree*, *projection*, *dispatch root*, *leaf action*, *synthetic affordance*, *borrowed input* - so that the ideas can be recognized outside Cobra.

Evidence discipline

The claims in this book use three kinds of evidence:

1. **Public interfaces and runtime code** show what the framework can express and how it behaves.
2. **Tests** reveal ordering, compatibility, and error contracts that may be easy to miss in comments.
3. **Repository history** is used for failure-derived lessons, especially when a bug fix states the violated invariant more clearly than the final code alone.

A source map at the end of each chapter points to the relevant Cobra files. Appendix C collects the complete map.

Design rule - Read implementations as contracts, not as recipes. The purpose of studying Cobra is not to copy every field or global switch. A reusable pattern is an invariant that survives a change of language, API shape, or storage strategy. The book therefore distinguishes the general rule from Cobra's particular mechanism and from the mechanism's limitations.

Chapter 1 - Build a Semantic Command Model

A scalable command-line architecture begins before parsing and before execution. It begins with a model that says what commands exist, how they are related, which metadata belongs to each command, and which properties flow through the hierarchy. Once that model is explicit, routing becomes one interpretation of it rather than the structure itself.

Learning objectives

After this chapter, you should be able to:

- explain why a handler table is not a sufficient semantic model for a mature CLI;
- define a command tree, command path, root, leaf, projection, and invariant;
- build a small command tree with Cobra-like API signatures;
- trace root-owned dispatch to a selected leaf action;
- compute effective flags under persistence and local shadowing;
- identify drift caused by parallel command inventories.

1.1 Motivation: why a table of handlers stops scaling

Imagine that the first version of `forge` supports only two operations:

```
switch os.Args[1] {
case "init":
    runInit(os.Args[2:])
case "deploy":
    runDeploy(os.Args[2:])
default:
    printUsage()
}
```

This is a reasonable starting point. The code contains both the route and the action. The problem appears when the interface grows. A later version needs nested commands, aliases, local flags, global flags, command descriptions, examples, hidden operations, generated reference pages, shell completion, and deprecation messages. One response is to add more tables:

```
var handlers = map[string]Handler{...}
var helpText = map[string]string{...}
var completionWords = map[string][]string{...}
var docsMetadata = map[string]DocEntry{...}
```

Each table is individually simple. Their relationship is not. A command can be added to `handlers` and forgotten in `helpText`. A renamed flag can remain in completion. A hidden command can disappear from help but remain in generated documentation. The architecture now contains several competing answers to the question, “What is the command-line interface?”

The important problem is therefore not parsing. It is *semantic duplication*.

Definition - Semantic duplication. Semantic duplication occurs when the same user-visible fact - a command name, parent-child relation, flag, visibility rule, or description - is authored independently in more than one representation. The representations can disagree because no mechanism makes them one fact.

The way out is to model the interface once and derive the different behaviors from that model.

1.2 The command tree as a semantic model

Definition - Semantic model. A semantic model is a data structure that represents the meaning-bearing parts of a system. For a CLI, those parts include command identity, ancestry, arguments, flags, descriptions, lifecycle hooks, and executable behavior.

Definition - Handler. A handler is the function or object invoked to perform the selected command's application-facing work. A handler is only one part of a command: it does not by itself describe ancestry, validation, help, completion, or visibility.

Definition - Metadata. Metadata is data that describes another object or behavior. Command names, summaries, examples, aliases, and visibility markers are metadata because interpreters use them to route or present the command.

Definition - Command. A command is a named operation in the user-facing language of the program. It may be executable itself, may contain child commands, or may do both.

Definition - Command tree. A command tree is a rooted tree $T = (V, E, r)$ in which each node in V is a command, each edge in E means "is a direct subcommand of," and r is the unique root command.

Definition - Root, interior node, and leaf. The root has no parent. An interior node has at least one child. A leaf has no children. In command frameworks, a leaf usually represents the most specific operation selected by an invocation, although an interior node can also be runnable.

Definition - Command path. The command path of node v is the ordered sequence of command names from the root to v . If v is set under config, then $\text{path}(v) = [\text{forge}, \text{config}, \text{set}]$.

Figure 1.1 shows the first forge model.

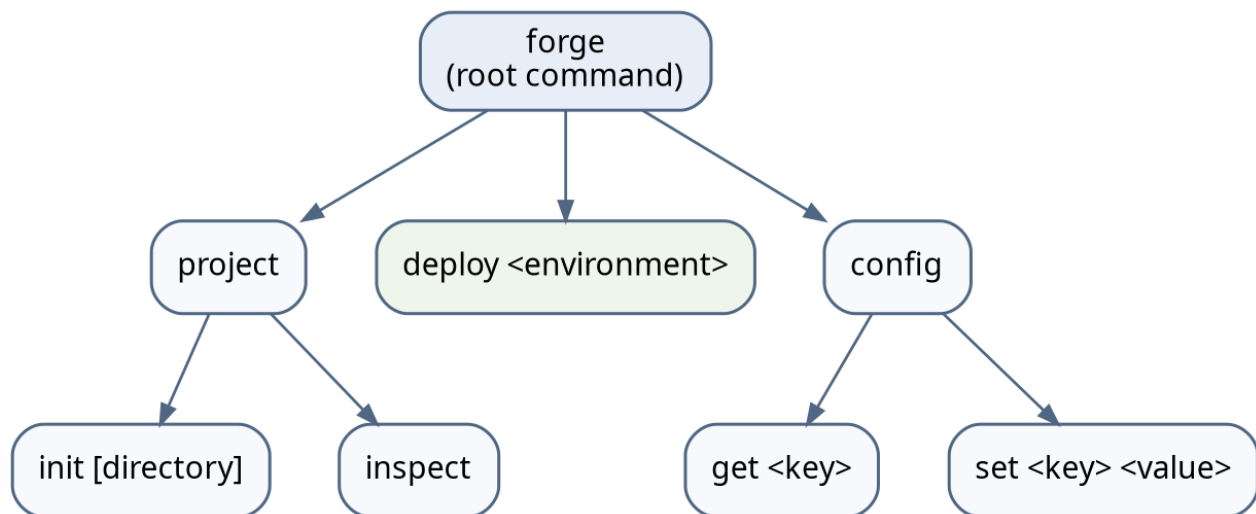


Figure 1.1 - The running example represented as a command tree.

The tree is semantic because the nodes contain more than function pointers. A simplified Cobra-like API might look like this:

```
type PositionalArgs func(cmd *Command, args []string) error

type Command struct {
    Use      string
    Aliases  []string
    Short    string
    Long     string
    Example  string
    Args     PositionalArgs
    Run      func(cmd *Command, args []string)
    RunE     func(cmd *Command, args []string) error
    Hidden   bool
    Deprecated string

    parent *Command
    commands []*Command
}

func (c *Command) AddCommand(children ...*Command)
func (c *Command) Parent() *Command
func (c *Command) Commands() []*Command
func (c *Command) Root() *Command
func (c *Command) CommandPath() string
```

The exact fields are less important than the ownership rule: the node that represents a command owns the metadata needed to interpret that command. The parent-child edges establish the interface's hierarchy.

Definition - Alias. An alias is an alternate spelling accepted for one canonical command.

Definition - Hidden command. A hidden command remains routable but is omitted from ordinary discovery.

Definition - Deprecated command. A deprecated command remains accepted for compatibility while warning users to migrate.

These are presentation and compatibility policies attached to one command identity, not separate commands.

Worked example: constructing forge

The following example is intentionally small. The handlers are placeholders; the point is the model.

```
root := &cobra.Command{
    Use:   "forge",
    Short: "Build and deploy projects",
}

project := &cobra.Command{
    Use:   "project",
    Short: "Manage project metadata",
}

initCmd := &cobra.Command{
    Use:   "init [directory]",
    Short: "Initialize a project",
    Args:  cobra.MaximumNArgs(1),
    RunE:  runProjectInit,
}
```

```

inspectCmd := &cobra.Command{
    Use:   "inspect",
    Short: "Inspect the current project",
    Args:  cobra.NoArgs,
    RunE:  runProjectInspect,
}

deployCmd := &cobra.Command{
    Use:   "deploy <environment>",
    Short: "Deploy a project",
    Args:  cobra.ExactArgs(1),
    RunE:  runDeploy,
}

configCmd := &cobra.Command{Use: "config", Short: "Read and write configuration"}
getCmd := &cobra.Command{Use: "get <key>", Args: cobra.ExactArgs(1), RunE: runConfigGet}
setCmd := &cobra.Command{Use: "set <key> <value>", Args: cobra.ExactArgs(2), RunE:
    ↪ runConfigSet}

project.AddCommand(initCmd, inspectCmd)
configCmd.AddCommand(getCmd, setCmd)
root.AddCommand(project, deployCmd, configCmd)

```

Adding `setCmd` does not merely make a handler reachable. It establishes the canonical path `forge config set`, places the node in the help hierarchy, gives completion a discoverable child, and gives documentation generation an object to traverse. That consequence motivates the next definition.

Student checkpoint. If a new command requires edits to four independent command-name lists, the architecture does not yet have a semantic spine.

Definition - Semantic spine. A semantic spine is one authoritative model that several behaviors interpret. For Cobra, the command tree is a semantic spine because dispatch, help, completion, and documentation all read the same command identities and relationships.

1.3 Projections: one model, several interpreters

A **projection** maps the semantic model into a purpose-specific result. The word emphasizes that the result is a view of the model, not another independent model.

Definition - Projection. Given a semantic model T , a projection is a function that derives a view or behavior from T . Typical CLI projections include routing, help text, shell-completion candidates, and generated documentation.

Definition - Interpreter. An interpreter is the component that performs a projection. It reads the model according to rules appropriate to one purpose.

We can write the major projections as functions:

$$\begin{aligned}
 \text{route} &: (T, \text{argv}) \rightarrow (v, \text{argv}_{\text{remaining}}) \\
 \text{help} &: T \rightarrow \text{Text} \\
 \text{complete} &: (T, \text{argv}_{\text{partial}}) \rightarrow (\text{Candidates}, \text{Directive}) \\
 \text{docs} &: T \rightarrow \text{DocumentTree}
 \end{aligned}$$

Figure 1.2 illustrates the fan-out.

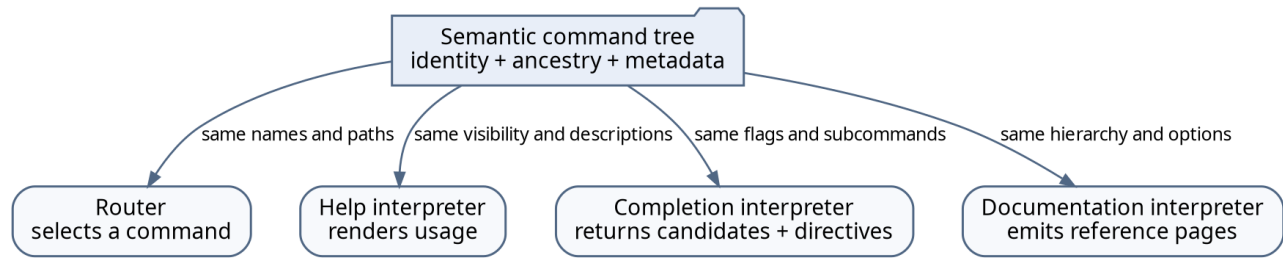


Figure 1.2 - Several interpreters consume the same semantic tree.

The projections are not required to produce identical output. Help may omit hidden commands. Completion may omit deprecated commands and aliases. Documentation may include long descriptions not shown in compact usage. The consistency requirement is narrower and more useful: the projections should obtain identities, ancestry, flags, descriptions, and visibility metadata from the same source.

Definition - Invariant. An invariant is a property that must remain true across valid states or executions of a system. In this chapter, invariants state what every projection must preserve even though its output format differs.

Four invariants capture the semantic-spine pattern:

1. **Identity invariant.** The canonical identity of a command comes from the tree used by dispatch.
2. **Ancestry invariant.** Parent-child relationships used by help, completion, and docs are the relationships used to route execution.
3. **Visibility invariant.** Hidden, deprecated, or help-topic status is interpreted from command metadata rather than copied into separate catalogs.
4. **Extension invariant.** Attaching a new command to the tree is the primary act that makes it available to projections.

Worked example: adding project archive

Suppose `forge` gains a command that archives local project metadata:

```
archiveCmd := &cobra.Command{
    Use:      "archive",
    Short:    "Archive local project metadata",
    Args:     cobra.NoArgs,
    RunE:     runProjectArchive,
    Deprecated: "use 'forge project export' for portable archives",
}
project.AddCommand(archiveCmd)
```

A tree-based architecture lets each interpreter discover the node. Dispatch can route `forge project archive`. Help can show a deprecation notice or hide it from the normal list according to policy. Completion can decline to suggest it while still accepting the explicit spelling. Documentation generation can record the command and its status. The command's presence and path are authored once; each projection decides how to present the shared fact.

Counterexample: a false single source of truth

A common mistake is to call a structure the “single source of truth” while allowing other structures to carry independent semantics. Consider:

```

type Route struct {
    Path    []string
    Handler Handler
}

type HelpEntry struct {
    Path        []string
    Description string
    Hidden      bool
}

```

If `Route.Path` and `HelpEntry.Path` are both handwritten, there are still two sources. The fact that both are generated from the same source file or constructed in the same function does not remove semantic duplication. The stronger design is to let help refer to the route node, or to generate both from a more primitive shared declaration.

Common misconception - “One file” is not “one model.” Co-location reduces search cost, but it does not establish identity. The architectural question is whether two components point to the same semantic object or independently restate its facts.

1.4 Root-owned dispatch and leaf-owned action

A command tree answers what operations exist. Execution still needs a rule for where routing begins and where the selected operation takes control.

Definition - Dispatch. Dispatch is the process of interpreting command-line tokens against the command tree to select one command node and determine which tokens remain as that command’s flags and positional arguments.

Definition - Orchestration root. The orchestration root is the unique node responsible for global execution setup, route selection, context propagation, and top-level error/usage policy.

Definition - Leaf action. A leaf action is the selected command’s local execution pipeline: parse applicable flags, validate its arguments and constraints, run its hooks, and invoke its handler.

Cobra’s `ExecuteC` re-anchors execution at the root even when it is called on a child command. Conceptually, the algorithm is:

```

function ExecuteC(receiver, environment):
    root = receiver.Root()
    if root != receiver:
        return ExecuteC(root, environment)

    ensure framework-provided affordances exist
    args = environment.args or process argv
    (selected, remaining) = resolve(root, args)
    selected.context = selected.context or root.context

    error = executeLeaf(selected, remaining)
    apply root/leaf error and usage policy
    return (selected, error)

```

The selected command then runs a separate pipeline:

```

function executeLeaf(command, args):
    parse effective flags
    handle help and version

```

```
require command to be runnable
validate positional arguments
run persistent and local pre-hooks
validate required flags and flag groups
invoke RunE or Run
run local and persistent post-hooks
```

Figure 1.3 shows the separation.

This division prevents an interior node from creating a second routing universe. Calling `ExecuteC` on `deployCmd` does not mean “interpret `argv` as if `deploy` were the root.” It means “execute the application whose root contains `deploy`.”

Worked trace: routing a deployment

Consider:

```
forge --profile team-a deploy production --image api:v42 --region us-east-1
```

A simplified trace is:

1. The root obtains `argv` and the execution context.
2. The resolver recognizes `--profile` as a root-persistent flag and `deploy` as a child command.
3. The resolver selects `deployCmd` and leaves `production --image api:v42 --region us-east-1` for the leaf pipeline.
4. The leaf assembles its effective flags, which include the inherited `--profile` flag and its own deployment flags.
5. The leaf validates the positional environment, flag constraints, and required flags.
6. The leaf runs its lifecycle and returns an error or success.
7. The root-level execution policy decides whether to print the error and usage text.

Notice that “root-owned” does not mean “root performs all work.” The root owns the control plane. The leaf owns the operation.

Why the distinction matters

The distinction gives the system one place for global concerns without forcing business logic into the root. Cancellation, default command injection, route selection, and error presentation can be centralized. Authentication, deployment planning, and project mutation remain in leaf handlers or domain services.

This pattern also makes tests more precise. A routing test can assert which leaf was selected. A leaf test can assert its validation and handler behavior. An integration test can exercise both through the root.

Counterexample: self-dispatching subtrees

Suppose every command can independently parse all remaining tokens and invoke a child:

```
root parses until it sees "project"
project reparses the original argv until it sees "init"
init parses again and runs
```

This design duplicates parsing rules at each level. Global flags can be consumed inconsistently. Error messages depend on which node initiated execution. Context and I/O may be initialized more than once. The architecture has no unique owner for “what invocation are we executing?”

Self-dispatching subtrees can be appropriate in a plugin boundary where a child truly is a separate application. Inside one semantic tree, they create accidental complexity.

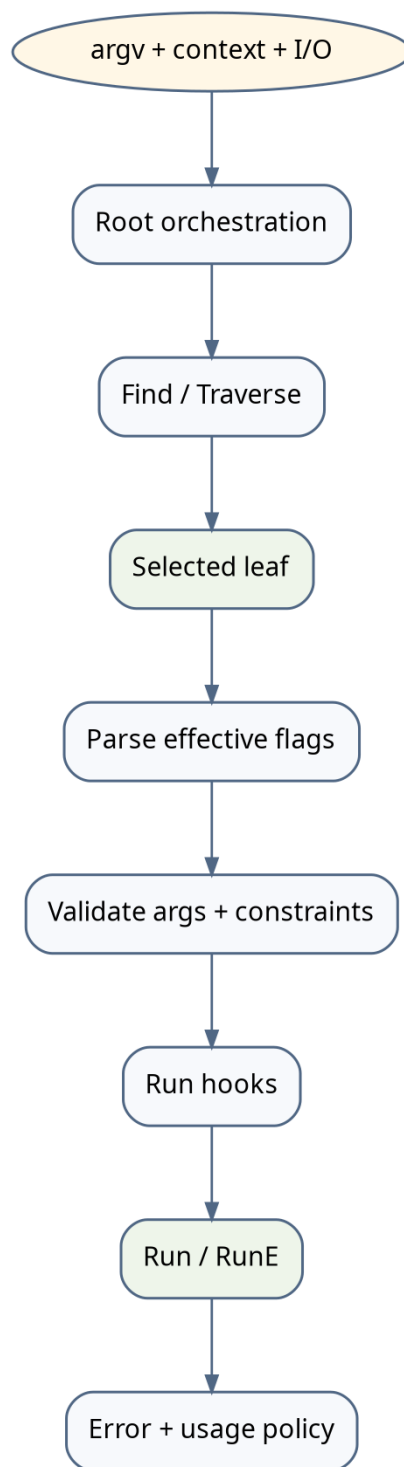


Figure 1.3 - The root owns routing; the selected leaf owns validation and action execution.

1.5 Hierarchical scope: persistent flags, local flags, and shadowing

The command tree can also express scope. Cobra distinguishes flags declared only for one command from flags that flow to descendants.

Definition - Local flag. A local flag is declared on command v and is available to v but not automatically to its children.

Definition - Persistent flag. A persistent flag is declared on command v and is inherited by descendants of v .

Definition - Effective flag view. The effective flag view $E(v)$ is the set of flags that command v can parse after combining local flags, its own persistent flags, and inherited persistent flags.

Definition - Shadowing. Shadowing occurs when a declaration in a nearer scope uses the same name as an inherited declaration and takes precedence in the effective view.

Let $A(v)$ be the ancestors of v , ordered from the root toward the parent. A first approximation is:

$$E(v) = L(v) \cup P(v) \cup \bigcup_{a \in A(v)} P(a)$$

This union hides an important detail: names can collide. The actual rule is a nearest-declaration rule. For each flag name n , the declaration chosen for $E(v)[n]$ is the nearest local or persistent declaration visible from v . A local flag on v can therefore shadow a persistent flag inherited from an ancestor.

Worked example: root configuration and local timeout

The root defines three persistent flags:

```
root.PersistentFlags().String("config", "", "configuration file")
root.PersistentFlags().String("profile", "default", "credential profile")
root.PersistentFlags().Bool("verbose", false, "verbose diagnostics")
```

The deploy command defines local flags:

```
deployCmd.Flags().String("image", "", "container image reference")
deployCmd.Flags().String("region", "", "target region")
deployCmd.Flags().Duration("timeout", 10*time.Minute, "deployment timeout")
```

The project inspect command also defines a local `--timeout`, but with a different meaning and default:

```
inspectCmd.Flags().Duration("timeout", 2*time.Second, "inspection deadline")
```

The two local flags do not conflict because their scopes do not overlap. Both commands also inherit `--config`, `--profile`, and `--verbose` from the root.

Now suppose the root later adds a persistent `--timeout=30s` for network operations. The local `deploy --timeout=10m` should remain the deployment timeout at the deploy node. The local declaration shadows the inherited one. Without a shadowing rule, the framework would either reject a useful specialization or silently select an arbitrary flag.

A useful way to inspect the result is as a table:

Command	Local flags	Inherited persistent flags	Effective <code>--timeout</code>
forge	none	none	root persistent 30s
forge project inspect	local 2s	root flags	local 2s
forge deploy	local 10m	root flags	local 10m

Command	Local flags	Inherited persistent flags	Effective --timeout
forge config get	none	root flags	inherited 30s

Scope is an architectural tool

Persistent flags are often described as a convenience for “global options.” That description understates the pattern. Hierarchy provides a scoped declaration mechanism. The same mechanism can support configuration, I/O, contexts, policies, or middleware. An ancestor establishes a default or capability; a descendant inherits it unless it supplies a nearer declaration.

This resembles lexical scope in programming languages:

```
outer declaration visible in inner scope
inner declaration with same name shadows outer declaration
sibling scopes remain independent
```

Fundamentals - Lexical scope. In a lexically scoped language, the meaning of a name is determined by the nested region where the name appears. A declaration in an inner region can shadow a declaration in an outer region. Command hierarchy is not source-code nesting, but the lookup rule is analogous.

Counterexample: “global” means mutable singleton

A poor implementation of global flags stores every option in a package-level variable:

```
var verbose bool
var timeout time.Duration
```

Every command reads the same variable. A command cannot specialize the flag safely. Tests can leak values into one another. Two embedded executions cannot use different environments. The term “global flag” has been confused with “global mutable state.” Persistent flags provide broad *scope* without requiring a singleton storage model.

1.6 A complete worked design review

Assume a team proposes the following architecture for forge:

- a map from command strings to handlers;
- a handwritten `usage.txt` file;
- a shell-completion script with a fixed list of words;
- global package variables for `--profile` and `--verbose`;
- each handler parses its own slice of `os.Args`.

We can review the design using the chapter’s vocabulary.

First, there is no command tree. The handler map provides route keys but not ancestry as a first-class relation. Second, help and completion are parallel inventories, so command identity is semantically duplicated. Third, every handler is partly a dispatcher because it chooses how to interpret process arguments. Fourth, global flags are represented as singleton state rather than hierarchical scope. Fifth, no invariant connects routing visibility, completion, and documentation.

A revised architecture introduces a root command and child nodes, attaches metadata and handlers to those nodes, gives root execution sole ownership of route selection, and declares `--profile` and

--verbose as persistent root flags. Help and completion become interpreters over the tree. The revised design has more explicit structure but fewer independent facts.

The key measure is not line count. It is the number of facts that must be synchronized manually.

Design rule - Minimize independent semantic facts. A mature architecture can contain many projections and adapters while still having a small semantic core. Complexity is reduced when new behavior interprets existing facts rather than introducing another inventory that must be kept consistent.

1.7 Limits of the command-tree pattern

A command tree is not the application's domain model. The node deploy should describe the user-facing operation and connect it to a handler. It should not become the authoritative store for deployments, users, permissions, transactions, or cluster state.

The tree is also not necessarily immutable. Cobra creates some default help and completion nodes late, and applications can add or remove commands. "One semantic model" means one authoritative graph, not one frozen value. Chapter 4 returns to the consequences for concurrency and compatibility.

Finally, not every interface is naturally a tree. A command may conceptually belong to several categories, or a plugin can expose operations whose ownership crosses subsystem boundaries. In those cases, the execution model may remain a tree while documentation adds tags or cross-links. Forcing every relation into the parent-child edge produces an overloaded model.

1.8 Chapter summary

A CLI becomes easier to extend when its commands form a semantic tree. The tree gives command identity and ancestry one authoritative representation. Routing, help, completion, and documentation become projections interpreted from that model. One root owns dispatch and global execution setup; the selected leaf owns validation and action execution. The hierarchy also provides scope for persistent declarations and local shadowing.

The chapter's central law is:

Declare the command language once, then interpret it many ways.

Exercises

Exercise 1.1 - Identify semantic duplication

A program stores routes in one JSON file, help text in another, and completion words in a shell script. List at least five facts that are duplicated. For each fact, describe a drift failure visible to a user.

Exercise 1.2 - Draw a command tree

Design a command tree for a package manager with these operations: search, install, remove, repository add, repository remove, cache clean, and cache inspect. Mark the root, interior nodes, leaves, and full command path of each leaf.

Exercise 1.3 - Define projections

For the package-manager tree, specify the inputs and outputs of four projections: route, compact help, completion, and Markdown documentation. State two invariants shared by all four and one filtering rule that differs among them.

Exercise 1.4 - Trace root-owned dispatch

Trace the invocation below through root orchestration and leaf execution. Identify exactly where each token is interpreted.

```
forge --profile team-a config set api.endpoint https://api.example
```

Exercise 1.5 - Compute effective flags

Suppose the root has persistent flags `--verbose` and `--timeout=30s`. The `project` command adds persistent `--format=text`. The `project inspect` command adds local `--timeout=2s` and local `--format=json`. Compute $E(v)$ for the root, `project`, `project init`, and `project inspect`.

Exercise 1.6 - Find the false unification

A framework stores every possible property - command metadata, authorization rules, database transaction settings, and UI colors - in one `Command` object so that “there is only one model.” Explain why this is a false unification. Propose boundaries between the command model and other models.

Exercise 1.7 - API design

Write a minimal language-neutral API for constructing a command tree. Include operations for adding a child, finding the root, computing a path, and enumerating visible children. State what errors or invalid states the API should prevent.

Exercise 1.8 - Counterexample construction

Construct a case where a command can execute successfully but should not appear in ordinary help or completion. Explain how a shared semantic model supports this difference without duplicating command identity.

Cobra source map for Chapter 1

- [command.go](#): `Command`, `AddCommand`, `Find`, `Traverse`, `ExecuteC`, `CommandPath`, local/persistent/inherited flag views, help and usage projections.
- [command_test.go](#): child routing, aliases, context propagation, flag inheritance, and shadowing behavior.
- [completions.go](#): completion resolves and interprets the command tree.
- [doc/md_docs.go](#): recursive Markdown documentation projection.
- [README.md](#): the public command/argument/flag conceptual model.

Chapter 2 - Execute with Explicit Lifecycles and Contracts

A command tree says what operations exist. It does not yet say when configuration is loaded, when arguments are validated, which cross-cutting hooks run, or whether two flags may be used together. Those questions belong to the execution model.

The main design move in this chapter is to treat execution as a staged lifecycle with explicit contracts. Hooks become ancestry-aware middleware. Positional rules become first-class validator functions. Relationships among flags become declarative constraints. The result is an execution machine that can be traced, tested, and explained.

Learning objectives

After this chapter, you should be able to:

- define lifecycle phase, hook, persistent hook, validator, combinator, and declarative constraint;
- state the ordering of a Cobra-like command execution;
- explain the difference between legacy nearest-hook semantics and full ancestry traversal;
- model positional validation as functions and conjunction;
- express all-or-none, at-least-one, and at-most-one flag groups mathematically;
- trace a complete `forge deploy` execution and identify where each failure is detected.

2.1 Motivation: a handler call is not an execution model

A naive dispatcher ends with a function call:

```
handler(args)
```

A production CLI usually needs more phases:

- parse flags according to the selected command's scope;
- recognize framework-level help or version behavior;
- validate positional arguments;
- load configuration and credentials;
- establish tracing or logging context;
- validate required flags and relationships among flags;
- perform the action;
- flush metrics, release resources, or record audit information;
- decide how errors and usage text are presented.

If each handler invents this sequence independently, the interface becomes inconsistent. One command validates before loading configuration; another loads configuration before discovering malformed input. One command emits usage on a domain failure; another does not. Cleanup can be skipped on early returns.

Definition - Lifecycle. A lifecycle is an ordered sequence of phases through which one selected command execution passes. A lifecycle is architectural when its order and failure behavior are part of the framework's contract rather than incidental code layout.

Definition - Phase. A phase is a named step with a purpose, inputs, and possible outcomes. Parsing, positional validation, pre-hooks, action execution, and post-hooks are phases.

The benefit of naming phases is not bureaucracy. It gives developers a place to attach behavior and gives tests an order to assert.

2.2 A staged execution pipeline

A simplified Cobra execution can be described as the following ordered pipeline:

1. Initialize default help/version behavior as late as possible.
2. Parse the selected command's effective flags.
3. If help or version was requested, return through the framework path.
4. Require the command to be runnable.
5. Run framework initializers.
6. Validate positional arguments.
7. Run persistent pre-hooks according to traversal policy.
8. Run the selected command's local pre-hook.
9. Validate required flags.
10. Validate relationships among flag groups.
11. Run the selected command's action.
12. Run the selected command's local post-hook.
13. Run persistent post-hooks according to traversal policy.
14. Run framework finalizers on function exit.

This order contains several design decisions. Positional validation occurs before command hooks. Required-flag and flag-group validation occur after pre-hooks, which allows pre-hooks to participate in command setup but also means they must not assume that every flag relationship has already been accepted. Post-hooks run only after the action has run successfully far enough to reach them; separate finalizers are used for unconditional framework cleanup.

Definition - Failure boundary. A failure boundary is a point in the lifecycle after which later phases do not run. For example, an argument-validation error prevents pre-hooks and the action from running. A pre-hook error prevents the action and ordinary post-hooks from running.

A useful design exercise is to ask of every phase: “What may this phase assume, and what later phases does its failure suppress?”

Worked example: phase placement

Suppose `forge deploy` needs to load a profile, validate a region, create a trace span, and contact a deployment service.

A coherent placement is:

- root persistent pre-hook: load the selected profile and attach credentials to context;
- deploy local pre-hook: create a deployment trace span or resolve deployment-specific defaults;
- positional validator: require exactly one environment name;
- required-flag validation: require `--region`;
- flag-group validation: require exactly one of `--image` and `--artifact`;
- RunE: call the deployment service;
- local post-hook: record a command-level success metric;
- root persistent post-hook: flush telemetry.

The positional rule is not hidden inside profile loading. The flag relationship is not rediscovered inside the service call. Each rule is attached to the phase that owns it.

2.3 Hooks as ancestry-aware middleware

Definition - Hook. A hook is a callback inserted before or after the selected action. Cobra supports local hooks on the selected command and persistent hooks whose scope can include descendants.

Definition - Cross-cutting concern. A cross-cutting concern is behavior such as tracing, configuration loading, or metrics that applies to several commands rather than one domain operation.

Definition - Middleware. Middleware is code arranged around an inner operation so that it can prepare, observe, transform, or clean up the operation. The hook sandwich is ancestry-scoped middleware.

Definition - Local hook. A local pre-hook or post-hook belongs to one command and runs only when that command itself is selected.

Definition - Persistent hook. A persistent hook is attached to a command and may apply when that command or a descendant is selected.

Definition - Hook sandwich. A hook sandwich is an execution order in which outer-scope pre-hooks enter from root toward the leaf, the leaf action runs, and post-hooks unwind from leaf toward the root.

For a selected path

$$r \rightarrow a_1 \rightarrow a_2 \rightarrow \dots \rightarrow \ell$$

full traversal produces:

$$pre(r), pre(a_1), \dots, pre(\ell), run(\ell), post(\ell), \dots, post(a_1), post(r)$$

Figure 2.1 renders the same idea procedurally.

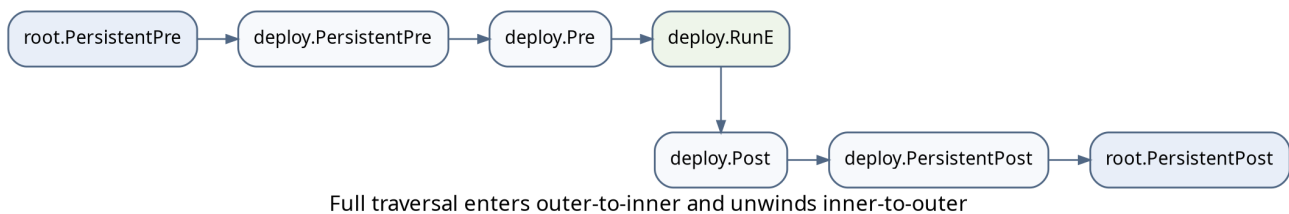


Figure 2.1 - Full hook traversal forms an outer-to-inner, then inner-to-outer sandwich.

This structure resembles nested middleware, lexical scopes, and stack unwinding. An outer pre-hook can establish an invariant that all inner operations observe. The corresponding post-hook can close or report on that scope after the inner action completes.

API shape

A simplified portion of Cobra's command API is:

```
type Command struct {
    PersistentPreRun func(cmd *Command, args []string)
```

```

PersistentPreRunE func(cmd *Command, args []string) error
PreRun           func(cmd *Command, args []string)
PreRunE          func(cmd *Command, args []string) error

Run              func(cmd *Command, args []string)
RunE             func(cmd *Command, args []string) error

PostRun          func(cmd *Command, args []string)
PostRunE         func(cmd *Command, args []string) error
PersistentPostRun func(cmd *Command, args []string)
PersistentPostRunE func(cmd *Command, args []string) error
}

```

The E variants return errors. When both forms exist for the same phase, the error-returning form takes precedence.

Compatibility-sensitive traversal

An important Cobra detail is that full ancestry traversal is not unconditional. The package-level switch:

```
var EnableTraverseRunHooks = false
```

preserves legacy behavior by default. With traversal disabled, Cobra walks from the leaf upward and runs only the first persistent pre-hook it finds; the same nearest-hook rule applies to persistent post-hooks. With traversal enabled, all persistent pre-hooks run root-to-leaf and all persistent post-hooks run leaf-to-root.

Definition - Compatibility mode. A compatibility mode preserves an older externally observable behavior while allowing an application to opt into a newer behavior. The existence of a switch means the behavior is part of the compatibility surface, not a private refactoring detail.

This distinction changes what a child command may assume. Under full traversal, a root pre-hook and a child pre-hook can both contribute setup. Under nearest-hook semantics, the child hook can suppress the root hook simply by existing.

Worked example: configuration and telemetry

Consider this setup:

```

root.PersistentPreRunE = func(cmd *cobra.Command, args []string) error {
    cfg, err := loadConfig(cmd.Flags().Lookup("config").Value.String())
    if err != nil { return err }
    cmd.SetContext(withConfig(cmd.Context(), cfg))
    return nil
}

root.PersistentPostRun = func(cmd *cobra.Command, args []string) {
    flushTelemetry(cmd.Context())
}

deployCmd.PersistentPreRunE = func(cmd *cobra.Command, args []string) error {
    span := startSpan(cmd.Context(), "deploy")
    cmd.SetContext(withSpan(cmd.Context(), span))
    return nil
}

deployCmd.PersistentPostRun = func(cmd *cobra.Command, args []string) {
    finishSpan(cmd.Context())
}

```

}

With full traversal, configuration loads before the deployment span starts, and the span finishes before root telemetry is flushed. The order is nested and intelligible.

With nearest-hook semantics, `deployCmd.PersistentPreRunE` can prevent the root configuration hook from running. If the handler expects configuration in context, the application fails. The code is not necessarily wrong; the *assumed traversal mode* is unstated.

A robust application either enables and tests full traversal or composes required setup explicitly into one persistent hook. It does not assume that “persistent” automatically means “all ancestors.”

Student checkpoint. Before placing behavior in a persistent hook, state which ancestors run in the application’s selected compatibility mode. If that statement is missing, the hook’s scope is ambiguous.

Counterexample: hooks as invisible business workflow

Hooks are appropriate for cross-cutting concerns whose scope follows the command hierarchy. They are a poor place for the core business workflow:

```
root pre-hook charges a credit card
child pre-hook creates an order
RunE merely prints success
```

The user-visible action has been scattered across invisible phases. Error recovery and retries become obscure. A better design keeps the domain transaction in a service called by `RunE`, while hooks prepare context, authentication, tracing, or other cross-cutting infrastructure.

Design rule - Hooks establish execution context; handlers own the domain action. This is a guideline, not a type-system guarantee. Violating it should require a deliberate reason.

2.4 Positional validation as first-class functions

The command `deploy <environment>` has a simple rule: exactly one positional argument is required. The rule should be independently named, testable, and reusable.

Definition - Validator. A validator is a function that examines a value and returns success or a diagnostic error without performing the command’s domain action.

Definition - First-class function. A function is first-class when it can be stored in a variable or field, passed as an argument, returned from another function, and composed like other values.

Cobra defines positional validation with a function type:

```
type PositionalArgs func(cmd *Command, args []string) error
```

A validator has the mathematical shape:

$$V : (Command, Args) \rightarrow Error \cup \{nil\}$$

`nil` means the arguments satisfy the rule. An error explains the first violated rule.

Cobra provides validators including:

```

func NoArgs(cmd *Command, args []string) error
func ArbitraryArgs(cmd *Command, args []string) error
func OnlyValidArgs(cmd *Command, args []string) error
func NoDuplicateArgs(cmd *Command, args []string) error
func MinimumNArgs(n int) PositionalArgs
func MaximumNArgs(n int) PositionalArgs
func ExactArgs(n int) PositionalArgs
func RangeArgs(min, max int) PositionalArgs
func MatchAll(validators ...PositionalArgs) PositionalArgs

```

Functions such as `ExactArgs(1)` are validator factories: they capture a parameter and return a validator.

Worked example: a release command

Suppose Chapter 3 later adds:

```
forge release promote <version> <environment>
```

The command requires exactly two arguments, accepts only known environment names in the second position, and rejects duplicate strings. We can compose small rules:

```

func knownEnvironmentAt(index int, known map[string]struct{}) cobra.PositionalArgs {
    return func(cmd *cobra.Command, args []string) error {
        if index >= len(args) {
            return nil // arity validator owns the missing-argument error
        }
        if _, ok := known[args[index]]; !ok {
            return fmt.Errorf("unknown environment %q", args[index])
        }
        return nil
    }
}

promoteCmd.Args = cobra.MatchAll(
    cobra.ExactArgs(2),
    cobra.NoDuplicateArgs,
    knownEnvironmentAt(1, environments),
)

```

Definition - Short-circuiting. Short-circuiting stops a composition as soon as its result is already known. `MatchAll` stops at the first validator error, so later validators do not run.

`MatchAll` implements conjunction with short-circuiting:

```

function MatchAll(v1, v2, ..., vn):
    return function(command, args):
        for validator in [v1, v2, ..., vn]:
            error = validator(command, args)
            if error != nil:
                return error
        return nil

```

If validators are predicates p_i , conjunction means:

$$MatchAll(p_1, \dots, p_n)(x) = p_1(x) \wedge \dots \wedge p_n(x)$$

The implementation returns the first diagnostic error rather than a Boolean. Ordering therefore affects

which error the user sees. Put structural rules such as arity before rules that index into arguments.

Fundamentals - Predicates and diagnostics. In mathematics a predicate maps a value to true or false. Software validators often enrich false with an error message. Composition still follows predicate logic, but error order becomes part of usability.

Why validation is separate from routing

Routing answers, “Which command does this token sequence name?” Positional validation answers, “Given that command, are the remaining nouns admissible?” Conflating them causes vague errors. If `forge deploy production extra` is routed to `deploy` and then rejected by `ExactArgs(1)`, the error can state that the command received too many arguments. A router that treats `extra` as an unknown subcommand may produce a misleading suggestion instead.

Counterexample: validation inside the handler

This code works but weakens the architecture:

```
RunE: func(cmd *cobra.Command, args []string) error {
    if len(args) != 1 {
        return fmt.Errorf("expected environment")
    }
    // perform deployment
}
```

The handler now mixes input contract and domain action. Completion and documentation cannot easily inspect the contract. Unit tests must enter the handler to test arity. More importantly, validation occurs later in the lifecycle and can run after hooks that should have been skipped for malformed input.

Custom domain validation can still occur in `RunE` when it requires network or business state. The design goal is to move cheap, structural admissibility rules into explicit validators.

2.5 Declarative relationships among flags

Individual flag types and required markers are not enough. Real interfaces contain relationships:

- `--username` and `--password` must be supplied together;
- at least one of `--image` and `--artifact` must be supplied;
- `--json` and `--yaml` cannot both be selected.

Definition - Declarative constraint. A declarative constraint records a relationship among inputs as metadata, separate from the imperative code that enforces it.

Definition - Flag group. A flag group is a named set of flags interpreted by one relationship rule.

Cobra exposes three group operations:

```
func (c *Command) MarkFlagsRequiredTogether(flagNames ...string)
func (c *Command) MarkFlagsOneRequired(flagNames ...string)
func (c *Command) MarkFlagsMutuallyExclusive(flagNames ...string)
```

Internally, Cobra records the relationships as flag annotations and later interprets them during validation and completion.

Mathematical definitions

Let G be a group of flag names and S the set of flags selected in one invocation.

All-or-none (required together):

$$|S \cap G| \in \{0, |G|\}$$

Either none of the group is selected or every flag in the group is selected.

At least one (one required):

$$|S \cap G| \geq 1$$

At most one (mutually exclusive):

$$|S \cap G| \leq 1$$

Combining at-least-one and at-most-one yields exactly one:

$$|S \cap G| = 1$$

Worked example: forge deploy

Declare the flags:

```
deployCmd.Flags().String("image", "", "container image reference")
deployCmd.Flags().String("artifact", "", "local artifact path")
deployCmd.Flags().String("username", "", "registry username")
deployCmd.Flags().String("password", "", "registry password")
deployCmd.Flags().Bool("json", false, "emit JSON")
deployCmd.Flags().Bool("yaml", false, "emit YAML")
deployCmd.Flags().String("region", "", "target region")
```

Then declare relationships:

```
deployCmd.MarkFlagsOneRequired("image", "artifact")
deployCmd.MarkFlagsMutuallyExclusive("image", "artifact")
deployCmd.MarkFlagsRequiredTogether("username", "password")
deployCmd.MarkFlagsMutuallyExclusive("json", "yaml")
_ = deployCmd.MarkFlagRequired("region")
```

The resulting behavior is easy to state:

Invocation fragment	Valid?	Reason
--image api:v42	yes	exactly one source selected
--artifact build/app.tar	yes	exactly one source selected
no source flag	no	at least one is required
both source flags	no	they are mutually exclusive
--username alice	no	password is missing
--username alice --password secret	yes	required-together group is complete
--json --yaml	no	output formats are mutually exclusive

The metadata is more valuable than equivalent if statements because more than one interpreter can use it. > **Definition - Runtime validation.** Runtime validation is the authoritative admissibility check performed when the program actually executes an invocation, regardless of whether completion or help was used.

Runtime validation rejects invalid states. Chapter 3 shows how completion can hide a mutually exclusive alternative or suggest missing companions.

Constraint metadata is not authorization

A valid flag combination is only syntactically and structurally admissible. It does not prove that the user may deploy to production or read a secret. Authorization belongs to the domain boundary and usually requires identity and external state.

Common misconception - “Valid” does not mean “permitted.” Framework validation protects the shape of a request. Domain authorization protects the effect of a request.

Counterexample: guidance without enforcement

Suppose completion stops suggesting `--yaml` after the user chooses `--json`, but runtime execution never validates the pair. The UI appears helpful, yet scripts can pass both flags and reach ambiguous behavior. Guidance is not authority. Every declarative relationship that matters for correctness must have a runtime validator; interactive assistance is a secondary projection.

2.6 Full worked trace: forge deploy

We can now trace a complete invocation:

```
forge --profile team-a deploy production \  
  --image registry.example/api:v42 \  
  --region us-east-1 \  
  --json
```

Assume full persistent-hook traversal is enabled.

Step 1: route selection

The root resolver selects `deployCmd`. It preserves `--profile` as an inherited root flag and passes the environment and deployment flags into the leaf pipeline.

Step 2: effective flag parsing

The leaf’s effective view contains root-persistent flags and deploy-local flags. Parsing records `profile=team-a`, `image=...`, `region=us-east-1`, and `json=true`.

Step 3: framework requests

Help and version flags are checked. Neither is selected, so execution continues. The command is runnable because `RunE` is defined.

Step 4: positional validation

`ExactArgs(1)` accepts `production`. If the user had supplied no environment or two environments, execution would stop before persistent command hooks.

Step 5: persistent and local pre-hooks

The root persistent pre-hook loads profile team-a and attaches credentials/configuration to context. A deploy persistent or local pre-hook opens a trace span and resolves deployment defaults.

Step 6: required flags and groups

--region is present. The source group contains exactly one of image and artifact. The output group contains at most one of json and yaml. The credentials group is empty, which is allowed under the all-or-none rule.

Step 7: leaf action

RunE constructs a domain request and calls a deployment service. > **Definition - Domain service.** A domain service implements application rules and effects that do not belong to CLI syntax, such as deployment authorization and cluster updates. > The service, not Cobra, checks whether the current identity may deploy to production.

Step 8: post-hooks and return policy

Local and persistent post-hooks record success and flush telemetry. If RunE returns an error, root/leaf policy decides whether Cobra prints the error and usage text.

The trace demonstrates why each concept needs a definition. Routing, validation, hook setup, domain authorization, and presentation policy are related but not interchangeable.

2.7 Failure modes and design tensions

Pre-hooks that mutate inputs

A pre-hook can technically change flag values or command state before required/group validation. This flexibility can be useful for defaults, but it can also make the declared interface misleading. Prefer deriving new execution context over rewriting what the user supplied. If mutation is necessary, test the post-hook validation state explicitly.

Post-hooks are not unconditional cleanup

Ordinary post-hooks do not necessarily run after every early failure. Resources that must always be released should use language-level defer, structured cleanup in the handler/service, or a framework finalizer whose semantics guarantee execution.

Validator order changes diagnostics

MatchAll(ExactArgs(2), knownEnvironmentAt(1)) is safe. Reversing the order may cause an index error or an “unknown environment” message when the real problem is a missing argument. Validator composition has an algebraic meaning, but user-facing error priority is also part of the design.

Global compatibility switches broaden the test matrix

A package-level hook-traversal switch means libraries embedded in the same process share one mode. Tests should reset it after use. Applications that expose both modes must test both, because lifecycle assumptions can differ.

2.8 Chapter summary

A predictable CLI executes through named phases. Ancestry-aware hooks provide a middleware sandwich when their traversal policy is explicit. Positional contracts become first-class validator functions and can be composed by conjunction. Flag relationships become declarative constraints expressed as all-or-none, at-least-one, and at-most-one rules. Runtime validation remains the authority; guidance and documentation can project the same metadata later.

The chapter's central law is:

Make ordering and admissibility explicit before the domain action runs.

Exercises

Exercise 2.1 - Place each concern

For a command that downloads an artifact, place each concern in a lifecycle phase: parse destination path, validate one URL argument, load proxy configuration, authorize repository access, create a temporary file, verify checksum, rename into place, and emit metrics. Explain your placement.

Exercise 2.2 - Hook-order trace

Given path `root -> repository -> mirror` with persistent pre/post hooks on all three nodes and local pre/post hooks on `mirror`, write the exact order under full traversal. Then write the order under Cobra's default nearest-persistent-hook behavior.

Exercise 2.3 - Design a validator algebra

Define validators for a command `forge project label <name> [value]` with these rules: one or two arguments, the name must match `[a-z][a-z0-9-]*`, and name and value must differ. Show how validator order affects diagnostics.

Exercise 2.4 - Translate prose into set constraints

Express each rule with `|S ∩ G|`: exactly one output format; zero or all TLS certificate fields; at least one target cluster; no more than two diagnostic modes. Which rules can be expressed with Cobra's three built-in group types, and which require a custom validator?

Exercise 2.5 - Find the hidden workflow

Review a design where a root pre-hook opens a database transaction, a child pre-hook writes domain records, RunE sends an email, and a post-hook commits. Identify failure and retry problems. Redesign the domain action so the transaction boundary is explicit.

Exercise 2.6 - Validation versus authorization

For `forge deploy production`, list five structural checks appropriate for Cobra validators and five domain checks appropriate for the deployment service.

Exercise 2.7 - Property test

Write pseudocode for a property test asserting that an exactly-one flag group accepts every singleton subset of G and rejects the empty set and every subset of size greater than one.

Exercise 2.8 - Compatibility decision

Your framework currently runs only the nearest ancestor hook. Propose a migration plan to full traversal. Include opt-in mechanics, diagnostics, tests, documentation, and criteria for eventually changing the default.

Cobra source map for Chapter 2

- [command.go](#): leaf execution order, persistent/local hooks, required-flag and flag-group validation, error and usage policy.
- [cobra.go](#): EnableTraverseRunHooks, initializers, and finalizers.
- [args.go](#): PositionalArgs, built-in validators, NoDuplicateArgs, and MatchAll.
- [flag_groups.go](#): flag-group annotations, validation algorithms, and completion projection support.
- [command_test.go](#): persistent-hook ordering and execution behavior.
- [flag_groups_test.go](#): valid and invalid combinations, inherited flags, and deterministic first-error behavior.

Chapter 3 - Derive Assistance from the Model

A command-line interface is not complete when valid invocations execute. Users also need to discover commands, understand options, recover from mistakes, and complete partially typed input. These facilities are sometimes treated as decoration around the “real” program. In a mature CLI, they are additional interpreters of the same language.

This chapter develops two related ideas. First, framework-provided help, version, and completion behavior should be inserted late enough that application-defined behavior can override it. Second, shell completion is best designed as a protocol in which the shell asks the executable to interpret the partial command line. Both ideas preserve one semantic owner for the interface.

Learning objectives

After this chapter, you should be able to:

- define affordance, synthetic affordance, late binding, override, protocol, in-band request, and directive;
- explain why framework defaults should be materialized late and through ordinary model objects;
- trace Cobra’s hidden completion request from shell to executable and back;
- use bitmask directives to separate completion candidates from shell behavior;
- explain how flag constraints influence both rejection and interactive guidance;
- derive help and generated documentation from a command tree without creating a second inventory.

3.1 Motivation: assistance is executable behavior

Suppose the forge team maintains these artifacts separately:

- a routing tree in Go;
- a hand-written help page;
- a Bash completion script containing command and flag names;
- a Markdown reference manual;
- a list of typo suggestions.

The artifacts can disagree even if all are reviewed carefully. More subtly, static shell completion cannot easily answer dynamic questions such as:

- Which deployment environments exist for the selected profile?
- Which flags remain legal after `--json` has been chosen?
- Which subcommands are visible under the current command?
- Does this flag expect a file, a directory, a repository name, or an arbitrary string?

The executable already knows most of these answers. The design question is how to let assistance reuse that knowledge without taking control away from the application.

Definition - Affordance. An affordance is a feature that helps a user perceive or perform an action. In a CLI, help flags, help commands, version flags, completion commands, examples, suggestions, and generated documentation are affordances.

Affordances are behavior. They have routing rules, output channels, compatibility constraints, and override policy. Treating them as architecture makes them more predictable.

3.2 Late-bound synthetic affordances

Frameworks commonly provide default commands and flags. Cobra can synthesize a help flag, version flag, help command, hidden completion request, and user-facing completion command.

Definition - Synthetic affordance. A synthetic affordance is a command, flag, or related model object supplied by the framework rather than declared directly by the application.

Definition - Late binding. Late binding delays the selection or creation of behavior until enough context is available. In this case, the framework waits until execution or rendering is near, checks what the application already defined, and creates a default only when necessary.

Definition - Override precedence. Override precedence is the rule that determines whether application-defined behavior or framework-provided behavior wins when both address the same affordance.

The reusable pattern is:

```
function ensureAffordance(model, identity, makeDefault):
  if model already contains application-defined identity:
    keep it
  else:
    attach makeDefault() as an ordinary model object
```

Figure 3.1 shows the decision.

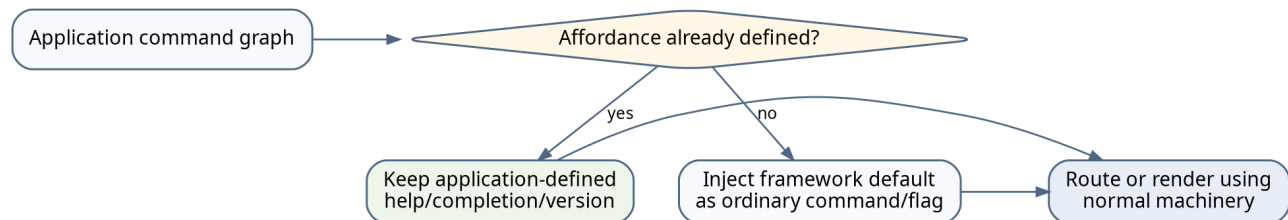


Figure 3.1 - Late binding preserves application overrides and then reuses ordinary routing/rendering.

Why “ordinary model object” matters

A default help command could be implemented as a special conditional outside the command tree. Cobra instead builds a Command with Use, Short, completion behavior, and a Run function, then attaches it to the tree. A default help flag is inserted into the command’s flag set. A completion command is also represented as a command.

This produces three benefits:

1. Existing routing and rendering machinery can process the default.
2. User-defined and framework-defined versions share a common shape.

3. The default participates in visibility, grouping, I/O, context, and command-path behavior without a parallel special-case system.

The default is synthetic in origin but ordinary after insertion.

Worked example: application-defined help

Suppose forge wants a domain-specific help command that opens a local manual when run interactively:

```
helpCmd := &cobra.Command{
    Use:   "help [command]",
    Short: "Open the Forge manual",
    RunE: func(cmd *cobra.Command, args []string) error {
        return openManual(cmd.Context(), args)
    },
}
root.SetHelpCommand(helpCmd)
```

Late-bound defaulting means the framework observes that a help command already exists and does not create a competing one. The application override wins by identity, not by initialization order accident.

The same principle applies to flags. A framework should not eagerly claim `-v` for version if the application already uses `-v` for verbose output. Cobra checks for conflicts and can add only the long `--version` form when the shorthand is occupied.

Side-effect containment

Synthetic commands can change the semantic graph. Adding a hidden completion command to a root that previously had no children can make the root appear to have subcommands and alter legacy argument behavior. Cobra contains this side effect by adding the hidden `__complete` command only while a completion request is being processed and removing it when the current invocation is not actually that request.

Design rule - Materialize defaults at the narrowest useful time and scope. Late creation is not valuable merely because it is late. It is valuable because the framework can inspect application choices and avoid changing unrelated executions.

Counterexample: eager defaults in a constructor

Imagine a framework constructor that always creates help, completion, `--help`, and `--version` before the application can configure the model. The application must remove defaults before defining replacements. Removal can leave cached paths or completion entries stale. A plugin loaded later can collide with a reserved name. Tests that construct a root for one narrow purpose unexpectedly receive extra children.

Eager creation can be correct when the default is truly mandatory and non-overridable. For extensible affordances, it converts a convenience into a source of conflicts.

3.3 Completion as an in-band protocol

Static completion scripts work well for fixed grammars. They work poorly when semantics live in the application. Cobra's solution is to let the shell script delegate semantic completion back to the executable.

Definition - Adapter. An adapter translates between two interfaces. A shell-completion adapter translates shell-specific cursor and insertion behavior into a request the executable understands, then translates the response back into shell behavior.

Definition - Completion candidate. A completion candidate is one textual continuation that the executable considers meaningful for the partial command line. It may carry a human-readable description.

Definition - Protocol. A protocol is an agreed sequence and format of messages exchanged between components.

Definition - In-band request. An in-band request travels through the application's ordinary invocation channel rather than a separate socket or service. Cobra uses hidden command names inside the normal argument vector.

Definition - Completion protocol. A completion protocol lets a shell adapter send a partial command line to the executable and receive candidate strings plus instructions describing how the shell should treat them.

Cobra reserves hidden request commands:

```
const ShellCompRequestCmd = "__complete"
const ShellCompNoDescRequestCmd = "__completeNoDesc"
```

A shell adapter can run a request such as:

```
forge __complete deploy pr
```

The executable resolves the real command context, computes completions, and writes a machine-readable response. Figure 3.2 shows the round trip.

This resembles a small local RPC, even though it is implemented with process invocation and standard streams.

Fundamentals - RPC without a server. Remote procedure call normally refers to invoking behavior in another process through a protocol. The processes do not need to be on different machines, and the callee does not need to be a long-running server. A shell launching a binary and exchanging structured stdout is still a protocol boundary.

3.4 The completion API: candidates plus directives

Cobra's completion function type is:

```
type Completion = string

type CompletionFunc = func(
    cmd *Command,
    args []string,
    toComplete string,
) ([]Completion, ShellCompDirective)
```

A completion string can contain a display description after a tab:

```
cobra.CompletionWithDesc("production", "Production cluster")
```

The return value contains two conceptually different products:

1. **Candidates** answer “Which textual continuations are meaningful?”
2. **A directive** answers “How should the shell behave after receiving them?”

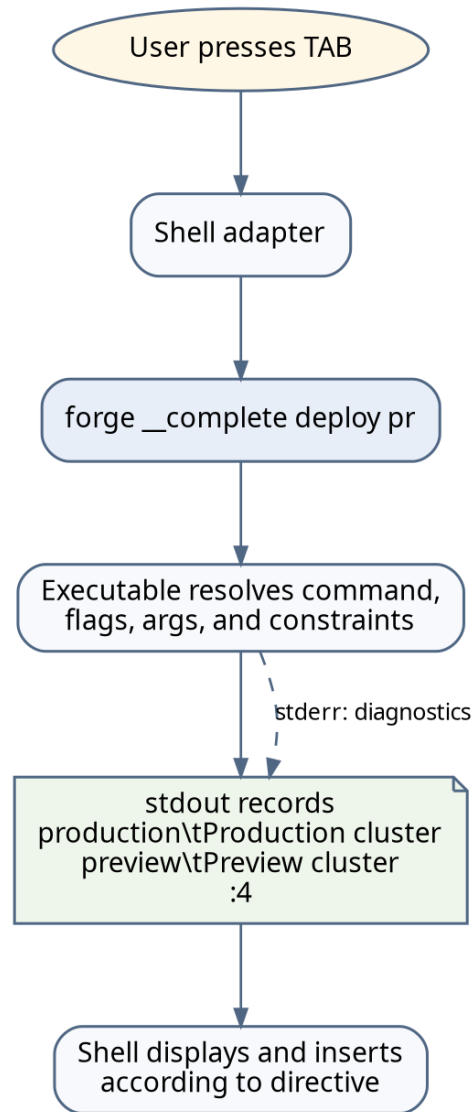


Figure 3.2 - The shell delegates semantic completion to the executable through a hidden command.

Definition - Directive. A directive is a control value returned with completion data that instructs the shell adapter to change behavior, such as suppressing file completion or avoiding a trailing space.

Definition - Bitmask. A bitmask is an integer whose individual bits represent independent Boolean options. Several completion behaviors can therefore be returned in one value.

Cobra represents directives as a bitmask. Simplified constants are:

```
type ShellCompDirective int

const (
    ShellCompDirectiveError ShellCompDirective = 1 << iota
    ShellCompDirectiveNoSpace
    ShellCompDirectiveNoFileComp
    ShellCompDirectiveFilterFileExt
    ShellCompDirectiveFilterDirs
    ShellCompDirectiveKeepOrder
)

const ShellCompDirectiveDefault ShellCompDirective = 0
```

If bit i represents behavior b_i , a combined directive is:

$$d = \sum_i x_i 2^i, \quad x_i \in \{0, 1\}$$

The adapter tests a behavior with bitwise conjunction:

$$behavior_i \text{ enabled} \iff (d \& 2^i) \neq 0$$

Worked example: completing environments

forge deploy can query environments available to the active profile:

```
deployCmd.ValidArgsFunction = func(
    cmd *cobra.Command,
    args []string,
    prefix string,
) ([]cobra.Completion, cobra.ShellCompDirective) {
    envs, err := environmentService.List(cmd.Context())
    if err != nil {
        return nil, cobra.ShellCompDirectiveError
    }

    var out []cobra.Completion
    for _, env := range envs {
        if strings.HasPrefix(env.Name, prefix) {
            out = append(out,
                cobra.CompletionWithDesc(env.Name, env.Description),
            )
        }
    }
    return out, cobra.ShellCompDirectiveNoFileComp
}
```

For prefix pr , the machine response can be conceptualized as:

```
production<TAB>Production cluster
preview<TAB>Preview cluster
:4
```

The last line encodes the directive. The exact integer is a protocol detail; the semantic meaning is “do not fall back to file completion.” The shell can display descriptions while inserting only the candidate text.

Worked trace: completing a flag name

Assume the user has typed:

```
forge deploy production --image api:v42 --r<TAB>
```

The shell adapter launches a hidden request containing the already typed tokens and the incomplete final token `--r`. The executable routes the prefix to `deploy`, assembles inherited and local flags, applies constraint metadata, and compares legal flag names with the prefix. `--region` matches. `--artifact` is excluded because `--image` is already selected and the two are mutually exclusive. The response contains `--region` with its description plus a directive suppressing ordinary file completion. The shell inserts `--region`; it does not need to know Cobra’s flag scopes or group annotations.

Student checkpoint. The shell owns cursor mechanics. The executable owns the meaning of the partial invocation. If the shell must reproduce command constraints, the adapter is too thick.

Fixed and dynamic completion

Cobra supports both fixed metadata and functions:

- `ValidArgs` provides a fixed list for positional completion.
- `ValidArgsFunction` computes candidates dynamically.
- flag completion functions can be registered for specific flags.
- subcommands and flag names are derived from the command model.

A fixed list is simpler and deterministic. A function can reflect external state but must be fast, cancellation-aware, and tolerant of partial input. Completion runs interactively; a two-second network request feels like a broken shell.

3.5 Protocol hygiene

A protocol implemented over `stdout` and `stderr` needs strict channel discipline.

Definition - Data channel. A data channel carries machine-consumed protocol output.

Definition - Diagnostic channel. A diagnostic channel carries human-oriented errors or debugging information that the protocol consumer can ignore.

Cobra writes completion candidates and the final directive to `stdout`. Diagnostic information goes to `stderr`. It also sanitizes candidate output: descriptions can be removed for no-description requests, only the first line is emitted, and surrounding whitespace is trimmed.

These rules prevent an innocent log line from becoming a fake completion candidate.

Protocol contract

A robust completion protocol should specify:

1. how the shell encodes the partially typed final token;
2. how command and flag context is resolved;
3. how candidates and optional descriptions are separated;
4. where the final directive appears;
5. which output stream is machine-readable;
6. how errors affect candidates and directives;
7. how protocol versions remain compatible.

Cobra’s final directive is deliberately the last stdout record so the shell can parse it unambiguously.

Counterexample: logging to stdout

Suppose a completion function calls a library that writes:

```
Connecting to cluster...
production
preview
:4
```

The shell may offer “Connecting to cluster...” as a candidate. The bug is not cosmetic; a data channel was contaminated. Libraries used during completion should accept an injected diagnostic writer or logger. Chapter 4 develops that boundary.

Counterexample: reimplementing application semantics in the shell

A hand-written shell script can parse known command words and flags. It becomes a second interpreter with a different language, release cycle, and test environment. It will not know application-defined validators, late-loaded plugins, profile-specific environments, or dynamically hidden options unless those rules are reimplemented.

A thin adapter is safer: the shell handles shell-specific insertion and display, while the executable owns semantic interpretation.

3.6 Constraint metadata drives both rejection and guidance

Chapter 2 encoded flag relationships as metadata. Completion can interpret the same relationships before execution.

For a required-together group, once one flag is selected, the remaining flags can be presented as required completions. For a one-required group, if none is selected, all candidates remain important. For a mutually exclusive group, once one flag is selected, the alternatives can be hidden from completion.

This is a **constraint projection**: one model produces both runtime rejection and interactive guidance.

Definition - Constraint projection. A constraint projection interprets declarative validity metadata for a secondary purpose, such as completion, form generation, or documentation, without replacing the authoritative validator.

Worked example: source and output flags

After the user types:

```
forge deploy production --image api:v42 --
```

completion should not suggest `--artifact`, because the source flags are mutually exclusive. It can still suggest `--region`, `--json`, `--yaml`, and other legal flags.

After the user chooses `--json`, completion should omit `--yaml`. If the user types `--username alice --`, completion can prioritize or require `--password` because the credentials group is incomplete.

The interface now guides the user toward valid states before the validator has to reject them.

Guidance remains subordinate to validation

Completion can be disabled, bypassed by scripts, or unable to represent a complex domain rule. Therefore:

$$Suggested(x) \not\Rightarrow Valid(x)$$

and

$$\neg Suggested(x) \not\Rightarrow \neg Valid(x)$$

A candidate can be valid but omitted for usability. A suggested candidate can become invalid by the time external state changes. Runtime validation remains the authority.

3.7 Help and documentation as projections

Cobra's default help and usage functions traverse command metadata, groups, flags, aliases, examples, and descriptions. The doc package recursively traverses the same tree to generate Markdown, manual pages, YAML, and reStructuredText.

A simplified documentation signature is:

```
func GenMarkdown(cmd *cobra.Command, w io.Writer) error
func GenMarkdownTree(cmd *cobra.Command, dir string) error
```

The tree generator recursively visits available child commands, computes names from command paths, and writes one page per node. Parent and child links are derived from the graph.

Definition - Generated reference projection. A generated reference projection turns the semantic command model into documentation that describes declared syntax and metadata. It does not prove the correctness of the handler's domain effects.

Worked extension: adding releases

Suppose the application adds:

```
forge release create <version>
forge release promote <version> <environment>
```

The team attaches two nodes under a new release node, supplies descriptions, validators, examples, and completion functions, and registers flags. From that one model change:

- routing recognizes both paths;
- ordinary help lists `release` and its children;
- completion suggests the new command names and arguments;
- generated Markdown gains pages and links;
- typo suggestions can consider the new names.

The handlers still need domain tests. The user-interface inventory does not need to be restated.

Suggestions are another projection

Cobra can suggest nearby command names using Levenshtein distance and prefixes. A suggestion function interprets the set of available command names; it does not change routing. This separation is important. A typo suggestion can say “Did you mean deploy?” without treating the typo as an implicit alias.

Fundamentals - Levenshtein distance. The Levenshtein distance between two strings is the minimum number of single-character insertions, deletions, and substitutions needed to transform one into the other. It is useful for suggestions, but thresholds must be conservative because an incorrect automatic match is worse than a missing hint.

3.8 Design tensions

The graph is authoritative but mutable

Late-bound affordances mutate the command graph. Introspection performed before execution can observe a different tree from introspection performed after default injection. Systems that need concurrent, immutable reads can compile a frozen snapshot rather than exposing the mutable builder directly.

Dynamic completion can trigger side effects

A completion function should query, not mutate, domain state. It may run frequently and unexpectedly as users press Tab. Treat it as a read-only, latency-sensitive operation with limited diagnostics.

Descriptions are part of a protocol record

A completion description cannot safely contain arbitrary newlines because each line has protocol meaning. Human-friendly rich text must be normalized at the boundary.

Generated docs can overstate certainty

Generated pages accurately reflect declared metadata at generation time. They do not verify authorization, remote state, or business semantics. Reference generation should be paired with examples and integration tests, not treated as executable proof.

3.9 Chapter summary

Assistance should be derived from the command model rather than maintained as a parallel interface. Framework defaults are safest when created late, only when absent, and as ordinary model objects. Shell completion can remain thin and portable by delegating semantic interpretation to the executable through a hidden in-band protocol. Candidates carry data; directives carry shell behavior. Constraint metadata can guide completion as well as reject invalid execution, while runtime validation remains authoritative.

The chapter’s central law is:

Let the executable explain its own language.

Exercises

Exercise 3.1 - Late-binding policy

Design override rules for a framework-provided help command, `--help` flag, completion command, and `--version` flag. For each, state when the default is created, how a collision is detected, and whether the user can disable it entirely.

Exercise 3.2 - Completion protocol grammar

Write an EBNF or line-oriented grammar for a completion response containing zero or more candidates, optional descriptions, and one final directive. Specify escaping or sanitization rules.

Exercise 3.3 - Bitmask reasoning

Assume `NoSpace` = 2, `NoFileComp` = 4, and `KeepOrder` = 32. Compute the integer for a directive that combines all three. Write pseudocode that tests whether `NoFileComp` is enabled.

Exercise 3.4 - Dynamic completion design

Design completion for `forge deploy <environment>` when environments come from a remote service. Address caching, cancellation, authentication errors, latency, stale data, and offline behavior.

Exercise 3.5 - Constraint projection

For the `forge deploy` flag groups in Chapter 2, specify the completion candidates after each partial invocation:

```
--image api:v42 --  
--artifact build/app.tar --json --  
--username alice --
```

Distinguish hidden candidates, prioritized candidates, and runtime-invalid states.

Exercise 3.6 - Thin versus thick adapters

Compare a static Bash script that knows the entire grammar with a thin adapter that invokes `__complete`. List advantages and costs of each. Identify one environment where static completion is still the better choice.

Exercise 3.7 - Documentation projection

Write pseudocode for generating one Markdown page per visible command. Include parent/child links, local flags, inherited flags, examples, and a stable filename rule. Identify two ways filename generation can collide.

Exercise 3.8 - Counterexample review

A completion function writes progress messages to `stdout`, modifies a configuration file to “warm the cache,” and returns every possible flag regardless of current constraints. Identify the protocol, ownership, and usability violations. Propose repairs.

Cobra source map for Chapter 3

- [command.go](#): late creation of help/version flags and the help command; help and usage projections.
- [completions.go](#): hidden request commands, `CompletionFunc`, completion directives, output protocol, dynamic resolution, and default completion command.
- [completions_test.go](#): command, flag, description, directive, traversal, and edge-case protocol behavior.
- [flag_groups.go](#): projection of required and mutually exclusive groups into completion state.
- [doc/md_docs.go](#): recursive Markdown generation and parent/child links.
- [doc/](#): Markdown, man, YAML, and reStructuredText documentation interpreters.

Chapter 4 - Testable Boundaries and Ownership-Safe Evolution

The first three chapters produced a semantic tree, an execution lifecycle, and several projections. The remaining question is whether the system can be executed deterministically outside a real terminal and evolved without violating hidden ownership or compatibility contracts.

CLI code naturally reaches for process globals: `os.Args`, `os.Stdin`, `os.Stdout`, `os.Stderr`, environment variables, and process-wide switches. These defaults are convenient at the binary boundary and hazardous inside a reusable engine. Cobra's design places overridable seams around several of them. A later completion bug also demonstrates that receiving a slice does not imply owning its backing storage.

Definition - Standard streams. Standard input, standard output, and standard error are the process's conventional byte streams for input, machine or user output, and diagnostics.

Definition - Context. In Go, a context carries cancellation, deadlines, and request-scoped values across API boundaries. It belongs to one execution, not to global application configuration.

Definition - Deterministic execution. Execution is deterministic for a test when controlled inputs and dependencies produce repeatable observable results without relying on ambient process state.

Learning objectives

After this chapter, you should be able to:

- define process boundary, dependency, seam, injected execution environment, ownership transfer, borrowed input, and backing array;
- build a deterministic command test with injected arguments, context, and I/O;
- explain ancestor fallback for process-facing dependencies;
- predict when Go's `append` reuses a backing array and can mutate caller-visible data;
- reconstruct the completion mutation defect and its repair;
- distinguish a mutable builder graph from an immutable execution snapshot;
- design a contract-oriented test suite for a command framework.

4.1 Motivation: process defaults are hidden dependencies

This handler is easy to write:

```
func runDeploy() error {
    environment := os.Args[2]
    fmt.Fprintln(os.Stdout, "deploying", environment)
    return nil
}
```

It is difficult to host. A unit test must replace process arguments or execute a subprocess. A GUI wrapper cannot redirect output without replacing a global stream. Two concurrent invocations cannot use different argument vectors. Cancellation is absent unless another global mechanism is added.

Definition - Dependency. A dependency is a value or service that a component reads from or writes to in order to do its work.

Definition - Process boundary. The process boundary is the interface between the command engine and process-level facilities such as argv, standard input, standard output, standard error, context/cancellation, environment, and exit status.

Definition - Seam. A seam is a place where a dependency can be substituted without changing the component's core logic.

The goal is not to ban process globals. The goal is to treat them as defaults selected at the outermost boundary, not as invisible dependencies scattered through the command graph.

4.2 An injectable execution environment

Cobra exposes setters and context-aware execution methods:

```
func (c *Command) SetArgs(args []string)
func (c *Command) SetContext(ctx context.Context)
func (c *Command) ExecuteContext(ctx context.Context) error
func (c *Command) ExecuteContextC(ctx context.Context) (*Command, error)

func (c *Command) SetIn(r io.Reader)
func (c *Command) SetOut(w io.Writer)
func (c *Command) SetErr(w io.Writer)

func (c *Command) InOrStdin() io.Reader
func (c *Command) OutOrStdout() io.Writer
func (c *Command) ErrOrStderr() io.Writer
```

When no override is supplied, Cobra falls back to the process defaults. Conceptually, one execution environment is:

$$Env = (argv, context, stdin, stdout, stderr)$$

Definition - Injected execution environment. An injected execution environment is an explicit set of process-facing dependencies supplied by a host or test for one run of the command graph.

Figure 4.1 shows the boundary.

Ancestor fallback

Cobra's internal input/output accessors first check the current command, then walk to a parent, then use the process default. This creates scoped dependency inheritance:

```
child override
  else parent override
    else root override
      else process stream
```

Definition - Ancestor fallback. Ancestor fallback is a lookup rule in which a node uses its own dependency override if present, otherwise the nearest ancestor's override, otherwise a system default.

This rule means a test can configure the root once and capture help, errors, completion output, and handler writes performed through Cobra's accessors by any selected child.

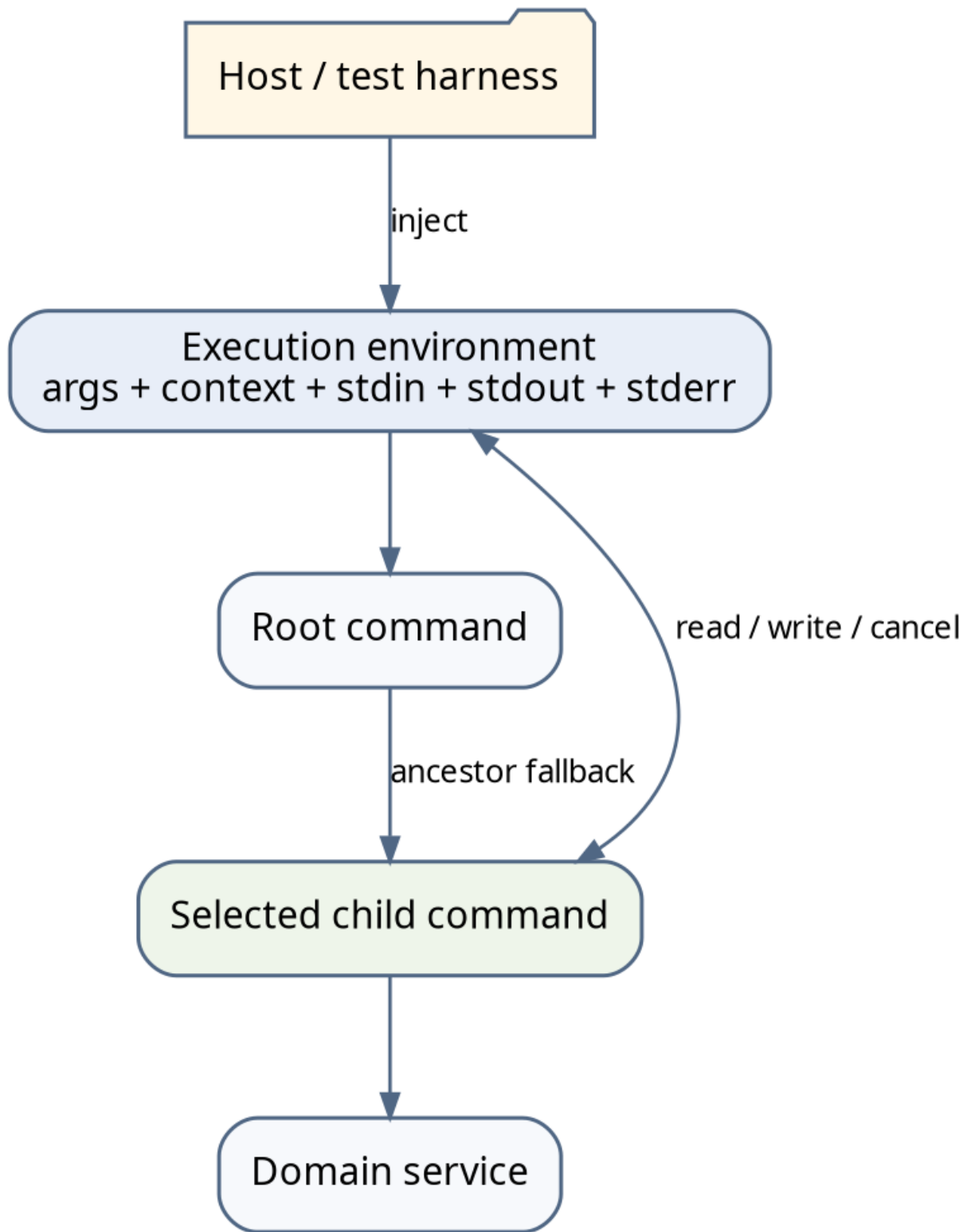


Figure 4.1 - A host injects one execution environment, and descendants inherit it through the command hierarchy.

Worked example: deterministic execution test

A small test harness can execute the real command tree without a subprocess:

```
func executeForTest(
    ctx context.Context,
    root *cobra.Command,
    args ...string,
) (selected *cobra.Command, stdout string, stderr string, err error) {
    var out bytes.Buffer
    var errOut bytes.Buffer
    root.SetOut(&out)
    root.SetErr(&errOut)
    root.SetIn(strings.NewReader(""))
    root.SetArgs(args)

    selected, err = root.ExecuteContextC(ctx)
    return selected, out.String(), errOut.String(), err
}
```

A route-and-action test becomes:

```
func TestDeployDryRun(t *testing.T) {
    root := newForgeCommand(fakeServices())
    selected, out, errOut, err := executeForTest(
        context.Background(),
        root,
        "deploy", "preview",
        "--image", "api:v42",
        "--region", "us-east-1",
        "--dry-run",
    )

    if err != nil { t.Fatal(err) }
    if selected.Name() != "deploy" { t.Fatalf("selected %q", selected.Name()) }
    if errOut != "" { t.Fatalf("unexpected stderr: %s", errOut) }
    if !strings.Contains(out, "plan") { t.Fatalf("missing plan: %s", out) }
}
```

The test is deterministic because it owns inputs and captures outputs. It also exercises routing, flag parsing, validation, hooks, and the handler together.

Worked example: embedding forge in a desktop application

A desktop host can reuse the same command graph without pretending to be a terminal. It supplies an argument vector assembled from UI controls, a context tied to the window's Cancel button, a reader for scripted input, and writers that append to an output panel. The host invokes `ExecuteContextC` and receives the selected command and error. No subprocess is required, and the CLI remains the authoritative parser for its own language.

This example also shows the limit of injection: a handler that calls `os.Exit`, writes directly to a process stream, or ignores context still escapes the host's boundary. Embeddability is an end-to-end property.

Context as a dependency carrier

`ExecuteContext` attaches a context to the root and propagates it to the selected command when the leaf has no context of its own. Handlers can retrieve it through `cmd.Context()`.

Context is appropriate for cancellation, deadlines, request-scoped values, and trace propagation. It is not a substitute for every dependency. Large service registries are usually clearer as explicit fields or closures captured when constructing commands.

Fundamentals - Cancellation is cooperative. Supplying a context does not forcibly stop work. The handler and every called service must observe `ctx.Done()` or use context-aware APIs. Injection provides the signal; application code must cooperate.

Boundary completeness

Capturing `cmd.OutOrStdout()` does not capture code that writes directly to `os.Stdout`. A seam is complete only when all relevant code uses it.

Bad:

```
fmt.Fprintln(os.Stdout, result)
```

Better:

```
fmt.Fprintln(cmd.OutOrStdout(), result)
```

Best for domain separation:

```
result, err := service.Deploy(cmd.Context(), request)
if err != nil { return err }
return encoder.Encode(cmd.OutOrStdout(), result)
```

The domain service returns data or errors; the CLI adapter owns presentation through the injected writer.

4.3 Process defaults versus constructor-enforced dependencies

Cobra uses optional overrides: if a value is absent, process state is used. This is convenient for applications because a root command can execute with little setup. It is less strict than a constructor that requires an explicit environment:

```
type Environment struct {
    Args    []string
    Stdin   io.Reader
    Stdout  io.Writer
    Stderr  io.Writer
    Ctx     context.Context
}

func Execute(root *Command, env Environment) error
```

The two styles make different tradeoffs.

Optional override	Required environment object
concise for ordinary binaries	dependencies always visible
backward compatible	easier concurrency reasoning
can fall back to globals unexpectedly	more construction ceremony
state stored on mutable command objects	environment naturally per execution

A reusable framework can expose both: a convenience `Execute()` that builds an environment from process defaults and a lower-level `ExecuteWith(env)` that has no hidden process dependencies.

Design rule - Put convenience at the edge. A convenience wrapper may read process globals. The core execution function should be explicit enough to test and embed.

4.4 Ownership: a slice value is not an owned array

Arguments introduce a second boundary question. When a caller passes `[]string`, may the framework mutate or append to it?

Definition - Ownership. Ownership is the responsibility and authority to mutate, retain, or dispose of a resource.

Definition - Ownership transfer. Ownership transfer is an explicit contract by which the caller gives the callee authority over a resource after the call.

Definition - Borrowed input. Borrowed input is data supplied for temporary use without transferring ownership. A callee may read it but must not perform caller-visible mutation unless the contract says otherwise.

Definition - Backing array. In Go, a slice refers to a contiguous backing array through a header containing a pointer, length, and capacity. Copying the slice value copies the header, not the elements.

A slice can be modeled as:

$$\text{Slice} = (p, \text{len}, \text{cap})$$

where p points to the first visible element. A sub-slice adjusts the header but usually shares the same backing array.

When append aliases

For a slice s , appending k elements can reuse the current backing array when:

$$\text{len}(s) + k \leq \text{cap}(s)$$

If capacity is sufficient, `append` writes after the visible length in the shared array. If capacity is insufficient, Go allocates another array and copies elements. Therefore the same source code can either mutate caller-visible storage or not, depending on capacity.

Figure 4.2 illustrates the risk.

Small worked example

```
base := make([]string, 3, 5)
base[0] = "forge"
base[1] = "deploy"
base[2] = "production"

borrowed := base[1:3]           // len=2, cap=4, aliases base
working := append(borrowed, "--")
```

Because `borrowed` has spare capacity, `working` reuses the backing array. `base[3]` now contains `--` even though the function appeared to append to a local variable.

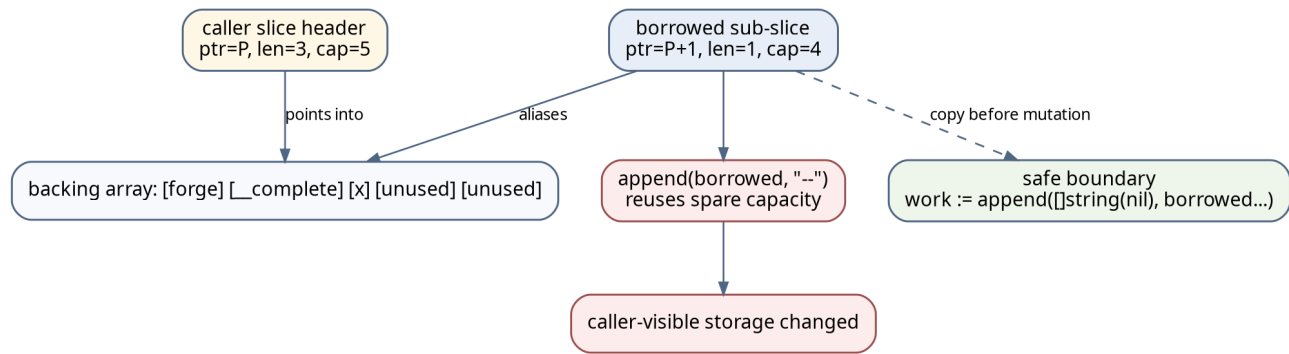


Figure 4.2 - A borrowed sub-slice can share spare capacity with caller-owned storage; append may mutate that storage.

If another part of the caller later reslices `base[:4]`, it observes the mutation. A more direct mutation can occur when a sub-slice begins earlier and `append` overwrites an element already visible through a different slice.

The safe copy-before-mutation patterns include:

```
working := make([]string, len(borrowed))
copy(working, borrowed)
working = append(working, "--")
```

or:

```
working := append([]string(nil), borrowed...)
working = append(working, "--")
```

The first form can document capacity intentionally:

```
working := make([]string, len(borrowed), len(borrowed)+1)
copy(working, borrowed)
working = append(working, "--")
```

4.5 Failure-derived lesson: completion mutated `os.Args`

In April 2026, Cobra fixed a completion defect recorded in commit [746ef07158728502482cea9f880a6f4b21ef29a9](#).

The relevant sequence was:

1. completion received arguments ultimately derived from `os.Args[1:]` or `SetArgs`;
2. traversal returned sub-slices sharing the same backing array;
3. completion temporarily appended `--` as a sentinel while checking interspersed flags;
4. when the sub-slice had spare capacity, `append` wrote into caller-owned storage;
5. code observing `os.Args` could see a real argument replaced by the sentinel.

The repair changed the initial trimming operation from a borrowed view to an owned copy:

```
trimmedArgs := make([]string, len(args)-1)
copy(trimmedArgs, args[:len(args)-1])
```

A regression test sets `os.Args`, enables child traversal, executes a completion request, and asserts that every original argument remains unchanged.

This is strong architecture evidence because it contains a violated invariant, a minimal fix, and a test that prevents regression.

Ownership law - Borrowed slices stay read-only. If an API receives a slice without explicit ownership transfer, it must treat the backing storage as immutable. Copy before append, sort, compact, reslice-and-write, or retention beyond the call.

Why the bug can evade tests

Capacity-dependent aliasing is unstable. A test may pass because `append` happens to allocate a new array. A production path may fail because traversal returns a slice with more capacity. Small changes to construction, input length, or compiler/runtime behavior can expose the bug.

A regression test should therefore construct the aliasing condition deliberately rather than hope ordinary inputs reproduce it.

Shallow copy and deep ownership

Copying `[]string` is enough because strings are immutable values from the application's perspective. For `[]map[string]string` or `[][]byte`, copying the outer slice still shares mutable inner objects.

Definition - Shallow copy. A shallow copy duplicates the outer container but preserves references held by its elements.

Definition - Deep copy. A deep copy recursively duplicates mutable referenced data to establish independent ownership.

The required depth depends on the contract. Do not deep-copy reflexively; define which layers are borrowed and which are owned.

4.6 Mutable graphs, execution snapshots, and compatibility

Definition - Concurrency. Concurrency means multiple executions or operations can make progress during overlapping time. Shared mutable command state requires synchronization or isolation when executions overlap.

Cobra command objects are mutable. Flags can retain parsed state, synthetic commands can be injected late, and package-level switches influence behavior. Injectable I/O makes testing easier but does not automatically make one command graph safe for concurrent execution.

Definition - Builder graph. A builder graph is a mutable object graph used to declare and configure the interface.

Definition - Execution snapshot. An execution snapshot is an immutable or isolated representation compiled from the builder graph for one run or for concurrent reads.

Counterexample: reusing one mutable root in parallel tests

Two tests share the same root command. One sets `SetArgs` and output buffer A while the other sets different args and output buffer B. Parsed flag state, writers, and the argument slice race. Each test can observe the other's configuration. Injectability did not create isolation because the injected values were stored on the shared mutable graph. The repair is to construct a fresh graph per test, serialize access, or compile an immutable snapshot whose execution environment is passed per request.

A framework designed for concurrent embedded execution might use this architecture:

```
mutable builder commands
  |
  | validate + compile
  v
immutable semantic snapshot
  |
  |--> execution request 1 with Env1
  |--> execution request 2 with Env2
  |--> help/docs projections
```

Cobra primarily exposes the mutable command graph directly. Applications should avoid concurrent execution on the same configured graph unless they establish stronger synchronization and state-reset guarantees.

Compatibility surfaces

Definition - Compatibility surface. A compatibility surface is any observable behavior on which users or applications may depend. Examples from the study include:

- whether all persistent hooks run or only the nearest one;
- command sorting order;
- case sensitivity and prefix matching;
- default help/completion injection;
- exact completion protocol records;
- error and usage emission policy.

A change can improve internal elegance and still break compatibility. The hook-traversal switch is a concrete example: full traversal is structurally attractive, but changing the default can alter configuration and cleanup behavior in existing applications.

Design rule - Evolve contracts explicitly. When behavior is observable, migrate it with opt-in modes, tests, diagnostics, and documentation rather than silently treating it as an implementation detail.

Counterexample: “fixing” hook traversal globally

A maintainer decides that all ancestor hooks should obviously run and changes the framework without a mode. An application previously relied on a child hook replacing root initialization. After the change, both run; configuration is loaded twice, a telemetry span is nested unexpectedly, and a cleanup hook closes a resource it did not open.

The new semantics may be better for new applications. The migration is still unsafe because the old semantics formed part of the contract.

4.7 Contract-oriented testing

A framework with several projections needs tests organized around invariants, not only functions.

Test categories

Category	Contract under test	Representative assertion
Model construction	tree identity and ancestry	every child has one parent; paths are stable
Routing	root selects one command	child execution re-anchors at root
Scope	effective flags and shadowing	nearest local declaration wins
Lifecycle	phase and hook order	expected pre/run/post sequence
Validation	admissibility algebra	invalid subsets are rejected deterministically
Projection	help/completion/docs share identities	newly attached visible command appears in each relevant view
Protocol	machine-readable completion	directive is last stdout record; diagnostics stay on stderr
Boundary	injected environment is honored	no process stream is touched in the test path
Ownership	caller input remains unchanged	completion does not mutate os.Args or SetArgs storage
Compatibility	modes remain intentional	legacy and full traversal both have tests

Property-style invariants

Some contracts are better expressed over many generated cases.

Tree path property:

```
for every node v except root:
    v.Parent().Commands() contains v
    v.CommandPath() = v.Parent().CommandPath() + " " + v.Name()
```

Exactly-one flag group property:

```
for every subset X of group G:
    valid(X) iff size(X) == 1
```

Borrowed-input property:

```
for generated args with extra capacity:
    original = clone(args)
    execute completion using args
    assert args == original
```

Projection identity property:

```
for every visible runnable node v:
    route(path(v)) selects v
    help(parent(v)) contains v.Name()
    completion(parent path + prefix(v)) can suggest v.Name()
    generated docs contain a page or entry for path(v)
```

The last property needs policy exceptions for hidden or deprecated commands. The test should encode those filters from metadata rather than maintain a second command list.

Golden tests and semantic tests

Definition - Property test. A property test checks a general invariant across many generated or systematically enumerated inputs rather than asserting only one example.

Definition - Golden test. A golden test compares complete rendered output with an approved reference artifact. It is strong for compatibility and formatting, but a small intended change can require updating a large expected file.

Golden output tests compare full help or completion text against a stored file. They are useful for formatting and compatibility but can be brittle. Semantic tests assert smaller properties: a command is present, a hidden command is absent, stderr is empty, or a directive bit is set.

Use both. Golden tests protect the user-visible contract. Semantic tests localize the reason for failure.

4.8 Capstone: assembling the forge architecture

The four chapters now fit together:

1. A mutable builder constructs one semantic command tree.
2. The root owns dispatch and receives an injected execution environment.
3. The selected leaf computes its effective flags, validates arguments and groups, and runs an explicit hook/action lifecycle.
4. Help, completion, suggestions, and documentation interpret the same command identities and meta-data.
5. Framework defaults are inserted late and only when the application has not supplied replacements.
6. Completion uses a hidden in-band protocol and keeps stdout machine-readable.
7. Runtime validation remains authoritative while completion projects constraints into guidance.
8. Borrowed argument slices are copied before any mutation-capable operation.
9. Tests assert routing, ordering, scope, projections, protocol records, boundary injection, and ownership.
10. Compatibility-sensitive behavior is represented as an explicit mode with tests for each mode.

This architecture is not “Cobra everywhere.” A different language can use immutable records, parser combinators, dependency-injected request objects, or a compiled grammar. The reusable achievement is the separation of responsibilities and the invariants connecting them.

4.9 Chapter summary

Process globals are useful defaults and poor hidden dependencies. Injected arguments, context, and I/O let the same command graph run in tests and embedded hosts. Ancestor fallback scopes those dependencies through the hierarchy. Slice inputs require an ownership contract because Go slice headers can alias caller-owned backing arrays; copy before mutation when ownership has not transferred. Mutable graphs and global compatibility switches broaden the reasoning and test surface, so observable behavior must evolve explicitly.

The chapter’s central law is:

Make the execution boundary explicit, and never confuse access with ownership.

Exercises

Exercise 4.1 - Boundary inventory

List every process-level dependency used by a typical CLI: arguments, streams, environment variables, working directory, signals, clock, random source, terminal width, exit status, and filesystem. Classify each as a value, service, or side effect. Propose seams for the five most important dependencies.

Exercise 4.2 - Build a test harness

Write a reusable test helper for a command framework that injects args, stdin, stdout, stderr, context, clock, and environment lookup. Decide which dependencies belong in one environment object and which should be constructor-injected services.

Exercise 4.3 - Ancestor fallback trace

The root sets stdout to buffer A. An interior config command sets stdout to buffer B. The leaf config get has no override. Another leaf deploy has no override. Determine where each leaf writes and explain the lookup rule.

Exercise 4.4 - Predict append behavior

For each slice, determine whether appending one element can reuse the backing array:

```
a := make([]string, 2, 2)
b := make([]string, 2, 5)
c := b[:1]
d := b[1:2:2]
```

State which caller-visible elements or capacity regions can be modified.

Exercise 4.5 - Design an ownership contract

Specify an API contract for `Parse(args []string)` under three alternatives: borrowed read-only input, ownership transfer, and copy-on-write. Discuss performance, retention, concurrency, and caller expectations.

Exercise 4.6 - Regression test construction

Write pseudocode for a test that reliably exposes append aliasing. The test must create spare capacity, pass a borrowed sub-slice, trigger a sentinel append, and assert the original array is unchanged.

Exercise 4.7 - Snapshot design

Design a compile step from mutable command builders to an immutable execution snapshot. List validation performed during compilation, data copied into the snapshot, caches that can be precomputed, and application behaviors that must remain late-bound.

Exercise 4.8 - Compatibility review

Choose one behavior - hook traversal, command sorting, prefix matching, or completion output - and write a compatibility plan for changing it. Include versioning, opt-in, deprecation, observability, and rollback.

Exercise 4.9 - Test matrix

Create a test matrix for `forge deploy` covering route selection, inherited flags, shadowing, validator errors, flag groups, hook order, dynamic completion, help output, injected I/O, cancellation, and input immutability. Mark which tests are unit, integration, property, golden, or regression tests.

Exercise 4.10 - Final design critique

A team claims its CLI is testable because it can replace `stdout`, but handlers still read `os.Args`, use `context.Background()`, call `time.Now()`, and write errors directly to `stderr`. Critique the claim and propose an incremental refactoring order.

Cobra source map for Chapter 4

- [command.go](#): `SetArgs`, context execution, stream setters/accessors, ancestor fallback, root-to-leaf context propagation, and mutable command state.
- [command_test.go](#): buffer-based execution helpers, context propagation, routing, and stream tests.
- [completions.go](#): current copy-before-mutation boundary in `getCompletions`.
- [completions_test.go](#): completion protocol coverage and the regression test protecting `os.Args` immutability.
- [Fix commit 746ef0715872](#): bug explanation, minimal copy fix, and regression test.
- [cobra.go](#): package-level compatibility switches and framework initializers/finalizers.

Appendix A - Glossary

This glossary collects the terms introduced in the four chapters. Definitions are intentionally shorter than the chapter explanations; use the chapter reference for motivation and examples.

Adapter. A component that translates between two interfaces, such as a shell and an executable completion protocol. See Chapter 3.

Alias. An alternate spelling accepted for one canonical command. See Chapter 1.

Bitmask. An integer whose bits independently represent Boolean options. See Chapter 3.

Completion candidate. A meaningful textual continuation for a partial command line. See Chapter 3.

Concurrency. Overlapping progress by multiple executions or operations, which requires care around shared mutable state. See Chapter 4.

Context. A Go value carrying cancellation, deadlines, and request-scoped values across API boundaries. See Chapter 4.

Cross-cutting concern. Behavior that applies to several commands rather than one domain action. See Chapter 2.

Deterministic execution. Execution whose controlled inputs and dependencies yield repeatable observable results. See Chapter 4.

Deprecated command. A command retained for compatibility while warning users to migrate to a replacement. See Chapter 1.

Domain service. Application code that owns business rules and effects rather than CLI syntax. See Chapter 2.

Golden test. A test comparing complete output with an approved reference artifact. See Chapter 4.

Handler. The function or object invoked to perform a selected command's application-facing work. See Chapter 1.

Hidden command. A routable command omitted from ordinary discovery. See Chapter 1.

Metadata. Data describing another object or behavior, such as a command summary or visibility marker. See Chapter 1.

Middleware. Code arranged around an inner operation to prepare, observe, transform, or clean it up. See Chapter 2.

Property test. A test of a general invariant across many inputs. See Chapter 4.

Runtime validation. The authoritative admissibility check performed during actual execution. See Chapter 2.

Short-circuiting. Stopping a composition as soon as the result is known. See Chapter 2.

Standard streams. The process's conventional input, output, and diagnostic byte streams. See Chapter 4.

Affordance. A feature that helps a user perceive or perform an action, such as help, completion, examples, or suggestions. See Chapter 3.

Ancestor fallback. A dependency lookup rule that uses a node's override, otherwise the nearest ancestor's override, otherwise a system default. See Chapter 4.

Backing array. The contiguous array referenced by a Go slice header. Several slices can share one backing array. See Chapter 4.

Borrowed input. Data supplied for use without transferring authority to mutate or retain it. See Chapter 4.

Builder graph. A mutable object graph used to declare and configure a command interface. See Chapter 4.

Command. A named operation in the user-facing CLI language. A command may be runnable, may own child commands, or may do both. See Chapter 1.

Command path. The ordered sequence of command names from the root to a node. See Chapter 1.

Command tree. A rooted tree whose nodes are commands and whose edges mean direct subcommand membership. See Chapter 1.

Compatibility mode. An explicit mode that preserves older observable behavior while allowing newer behavior to be selected. See Chapters 2 and 4.

Compatibility surface. Any observable behavior on which users or applications may depend. See Chapter 4.

Completion protocol. A protocol in which a shell sends a partial command line to the executable and receives candidates plus shell-behavior directives. See Chapter 3.

Constraint projection. Interpretation of validity metadata for a secondary purpose such as completion or form guidance. See Chapter 3.

Data channel. A channel reserved for machine-consumed protocol output. See Chapter 3.

Declarative constraint. A relationship among inputs recorded as metadata rather than embedded only in imperative enforcement code. See Chapter 2.

Deep copy. A copy that recursively duplicates referenced mutable data so that ownership is independent. See Chapter 4.

Dependency. A value or service that a component uses to do its work. See Chapter 4.

Diagnostic channel. A channel reserved for human-readable errors or debugging information that a protocol consumer can ignore. See Chapter 3.

Directive. A control value returned with completion candidates that tells the shell adapter how to behave. See Chapter 3.

Dispatch. Interpretation of command-line tokens against the command tree to select one command and determine its remaining inputs. See Chapter 1.

Effective flag view. The flags visible to a command after local declarations, persistent declarations, inheritance, and shadowing are combined. See Chapter 1.

Execution snapshot. An immutable or isolated representation compiled from a mutable builder graph for safe execution or concurrent reads. See Chapter 4.

Failure boundary. A lifecycle point at which an error prevents later phases from running. See Chapter 2.

First-class function. A function that can be stored, passed, returned, and composed like other values. See Chapter 2.

Flag group. A set of flag names interpreted by a relationship rule such as all-or-none or mutual exclusion. See Chapter 2.

Generated reference projection. Transformation of command metadata into reference documentation. See Chapter 3.

Hook. A callback inserted before or after a selected action. See Chapter 2.

Hook sandwich. An ordering in which outer pre-hooks run toward the leaf, the action runs, and post-hooks unwind toward the root. See Chapter 2.

In-band request. A request that travels through the application's ordinary invocation channel. See Chapter 3.

Injected execution environment. Explicit argv, context, and I/O supplied by a host or test for one run. See Chapter 4.

Interpreter. A component that reads a semantic model according to rules for one purpose, such as routing or documentation. See Chapter 1.

Invariant. A property that must remain true across valid system states or executions. See Chapter 1.

Late binding. Delaying behavior selection or object creation until enough context is available. See Chapter 3.

Leaf action. The selected command's local parse, validation, hook, and handler pipeline. See Chapter 1.

Lifecycle. The ordered phases through which one selected command execution passes. See Chapter 2.

Local flag. A flag visible to the command where it is declared but not automatically inherited by children. See Chapter 1.

Local hook. A pre- or post-hook that runs only when its own command is selected. See Chapter 2.

Orchestration root. The unique root responsible for global setup, route selection, context propagation, and top-level presentation policy. See Chapter 1.

Override precedence. The rule deciding whether application-defined or framework-provided behavior wins after a collision. See Chapter 3.

Ownership. Responsibility and authority to mutate, retain, or dispose of a resource. See Chapter 4.

Ownership transfer. An explicit contract that gives the callee authority over a resource after a call. See Chapter 4.

Persistent flag. A flag declared on a command and inherited by descendants. See Chapter 1.

Persistent hook. A hook attached to a command whose scope may include descendants. See Chapter 2.

Phase. A named lifecycle step with a purpose, inputs, and possible outcomes. See Chapter 2.

Process boundary. The interface between the command engine and process-level facilities such as argv and standard streams. See Chapter 4.

Projection. A function that derives a purpose-specific view or behavior from a semantic model. See Chapter 1.

Protocol. An agreed sequence and format of messages exchanged between components. See Chapter 3.

Semantic duplication. Independent authorship of the same meaning-bearing fact in more than one representation. See Chapter 1.

Semantic model. A data structure representing the meaning-bearing parts of a system. See Chapter 1.

Semantic spine. One authoritative model interpreted by several behaviors. See Chapter 1.

Seam. A place where a dependency can be substituted without changing core logic. See Chapter 4.

Shallow copy. A copy of an outer container that preserves references held by its elements. See Chapter 4.

Shadowing. A nearer declaration with the same name taking precedence over an inherited declaration. See Chapter 1.

Synthetic affordance. A command or flag supplied by the framework rather than directly declared by the application. See Chapter 3.

Validator. A function that checks admissibility and returns success or a diagnostic error without performing the domain action. See Chapter 2.

Appendix B - Selected Exercise Sketches

These are solution sketches, not exhaustive model answers. Several exercises admit multiple designs when the invariants are stated clearly.

Chapter 1

Exercise 1.3 - Projection invariants

For a package manager, all projections should derive canonical command names and parent-child paths from the same tree. Routing returns a selected node and remaining tokens. Compact help returns visible children and summaries. Completion returns candidates and shell directives. Documentation returns pages and links. A shared invariant is that `repository add` has the same canonical path everywhere. A differing rule is that completion may omit deprecated aliases while routing still accepts them.

Exercise 1.5 - Effective flags

With `root persistent --verbose` and `--timeout=30s`, `project persistent --format=text`, and `inspect-local --timeout=2s, --format=json`:

- `root`: `verbose`, `root timeout`;
- `project`: `verbose`, `root timeout`, `project format`;
- `project init`: the same effective set as `project`;
- `project inspect`: `verbose`, `local timeout=2s`, `local format=json`.

The `inspect-local` names shadow inherited declarations.

Chapter 2

Exercise 2.2 - Hook order

Under full traversal:

```
root persistent pre
repository persistent pre
mirror persistent pre
mirror local pre
mirror run
mirror local post
mirror persistent post
repository persistent post
root persistent post
```

Under nearest-persistent-hook behavior, only the nearest persistent pre and post found from the leaf run, plus the leaf's local hooks and action.

Exercise 2.4 - Set constraints

Exactly one output format is $|S \cap G| = 1$, expressible by combining one-required and mutually exclusive. Zero or all TLS fields is $|S \cap G| \in \{0, |G|\}$, expressible by required-together. At least one cluster is $|S \cap$

$|G| \geq 1$, expressible by one-required. No more than two diagnostic modes is $|S \cap G| \leq 2$, which requires custom validation because Cobra's built-in exclusivity bound is one.

Chapter 3

Exercise 3.3 - Bitmask

$2 + 4 + 32 = 38$. Test NoFileComp with:

```
if directive & 4 != 0:
    disable file completion
```

Exercise 3.5 - Constraint-aware completion

After `--image`, hide `--artifact`. After `--artifact --json`, hide `--image` and `--yaml`. After `--username alice`, prioritize or mark `--password` required. The runtime validator must still reject invalid direct invocations.

Chapter 4

Exercise 4.4 - Append behavior

`a` has no spare capacity, so appending one element allocates. `b` has spare capacity, so append can reuse its array. `c := b[:1]` has capacity 5 and aliases `b`, so append can overwrite `b[1]`. `d := b[1:2:2]` uses a full slice expression that limits capacity to one; appending allocates and prevents writes beyond the visible element.

Exercise 4.6 - Regression test

Construct an array with extra capacity, expose a shorter borrowed view, clone the full caller-visible state, invoke the code path that appends a sentinel, then compare every original element. The test must fail if the implementation merely re-slices instead of copying.

Appendix C - Cobra Source Map and Evidence Notes

Snapshot

Field	Value
Repository	spf13/cobra
Branch studied	main
Pinned commit	adbc8813901bba65827259daa8e22ff94ec1f30e
Commit date	2026-07-11
Failure-derived commit	746ef07158728502482cea9f880a6f4b21ef29a9
Analysis date	2026-08-16

Primary files

command.go

Primary evidence for the Command model, parent/child graph, root-owned ExecuteC, leaf execution, hook order, context propagation, stream and argument seams, flag inheritance and shadowing, default help/version behavior, and default help/usage projections.

[Open pinned command.go](#)

args.go

Primary evidence for the PositionalArgs function type, built-in arity and admissibility validators, NoDuplicateArgs, and MatchAll composition.

[Open pinned args.go](#)

flag_groups.go

Primary evidence for declarative all-or-none, at-least-one, and at-most-one flag groups, deterministic validation, and projection of constraints into completion behavior.

[Open pinned flag_groups.go](#)

completions.go

Primary evidence for hidden in-band completion commands, candidate/description formatting, bitmask directives, command and flag completion, late default completion injection, protocol hygiene, and copy-before-mutation of arguments.

[Open pinned completions.go](#)

cobra.go

Primary evidence for package-wide compatibility switches, especially persistent-hook traversal, plus framework initializer/finalizer registration and suggestion-distance support.

[Open pinned cobra.go](#)

doc/md_docs.go

Primary evidence for recursive Markdown generation from command paths, descriptions, examples, local/inherited flags, and parent/child links.

[Open pinned doc/md_docs.go](#)

Primary tests

- [command_test.go](#): routing, aliases, context propagation, I/O capture, flags, and hook ordering.
- [completions_test.go](#): completion candidates, descriptions, directives, traversal, and input immutability.
- [flag_groups_test.go](#): group validation, inherited flags, and deterministic errors.
- [doc/md_docs_test.go](#): Markdown generation behavior.

Failure-derived evidence

Commit [746ef0715872](#) explains how a sub-slice derived from caller arguments could share spare backing-array capacity and be mutated by a later append. The repair copies trimmed arguments into new storage and adds a regression test asserting that `os.Args` is unchanged.

Interpretation limits

The source proves Cobra's local behavior at the pinned snapshot. The broader architectural names in this book are abstractions derived from that evidence. Claims that a pattern is universally optimal would require comparison across additional frameworks and contexts. The book therefore presents tradeoffs, counterexamples, and adaptation guidance rather than universal prescriptions.